

KHOA AN TOÀN THÔNG TIN
HỌC VIỆN KỸ THUẬT MẬT MÃ



Đề tài:
XÂY DỰNG WEB TÍCH HỢP VỚI CHƯƠNG TRÌNH
TÌM KIẾM THEO ẢNH

Giáo viên hướng dẫn: Ths. Lê Bá Cường
Học Phần: Chuyên đề cơ sở

Sinh viên thực hiện:

Tạ Tùng Sơn	AT190144
Nguyễn Đức Chiến	AT190107
Nguyễn Văn Sơn	AT190143

Hà Nội, 2025

MỤC LỤC

MỤC LỤC.....	2
DANH MỤC TỪ VIẾT TẮT	4
DANH MỤC HÌNH ẢNH	6
LỜI NÓI ĐẦU	7
1. Lý do chọn đề tài.....	7
2. Mục tiêu thực hiện đề tài	8
CHƯƠNG 1. GIỚI THIỆU VỀ CÔNG NGHỆ TÌM KIẾM ẢNH TƯƠNG TỰ.....	9
1.1. Tổng quan trong xử lý ảnh.....	9
1.1.1. Giới thiệu về kỹ thuật Content – Based Image Retrieval (CBIR)	9
1.1.2. Các công đoạn trong xử lý ảnh	14
1.1.3. Mô hình học sâu (CNN) – ResNet.....	25
1.2. Mô hình trích xuất đặc trưng ảnh sử dụng mạng CNN	33
1.2.1. Cách trích xuất vector đặc trưng từ ảnh truy vấn	33
1.2.2. So sánh độ tương tự ảnh.....	39
1.3. Kết luận	46
CHƯƠNG 2. THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG TÌM KIẾM ẢNH THÔNG MINH	47
2.1. Tích hợp hệ thống CBIR vào ứng dụng web với mô hình ResNet – 50 và FAISS.....	47
2.1.1. Kiến trúc tổng thể của hệ thống	47
2.1.2. Lựa chọn và tối ưu hóa ResNet - 50	51

2.1.3. Tích hợp Faiss cho truy vấn ảnh quy mô lớn.....	52
2.2. Kiến trúc và triển khai ứng dụng Web.....	54
2.2.1. Thành phần giao diện người dùng (Frontend)	54
2.2.2. Thiết kế API Backend với Django.....	68
2.3. Kết luận.....	101
CHƯƠNG 3: TRIỂN KHAI THỰC NGHIỆM	102
3.1. Mô hình hệ thống.....	102
3.2. Triển khai thực nghiệm và đánh giá	105
3.2.1. Kết quả thực nghiệm.....	105
3.2.2. Kết luận và đánh giá	107
KẾT LUẬN.....	108
PHỤ LỤC.....	109
BẢNG PHÂN CÔNG CÔNG VIỆC	114
TÀI LIỆU THAM KHẢO	115

DANH MỤC TỪ VIẾT TẮT

STT	Từ viết tắt	Tên đầy đủ	Giải thích
1	CBIR	Content-Based Image Retrieval	Phương pháp tiếp cận trong lĩnh vực thị giác máy tính và truy xuất thông tin, nhằm tìm kiếm và truy xuất các hình ảnh từ cơ sở dữ liệu lớn dựa trên nội dung thị giác của chính hình ảnh đó
2	SIFT	Scale – Invariant Feature Transform	Là một trong những descriptor mạnh mẽ trong việc so sánh ảnh
3	SURF	Speeded Up Robust Features	Phiên bản tối ưu hơn của SIFT, nhanh hơn khoảng 3 – 5 lần nhưng vẫn đảm bảo được performance tương đương.
4	HOG	Histogram of Oriented Gradients	Phương pháp đặc biệt hiệu quả trong phát hiện vật thể và con người, ứng dụng trong phát hiện người và nhận dạng vật thể.

5	LBP	Local Binary Patterns	Một descriptor texture đơn giản nhưng hiệu quả trong việc so sánh ảnh.
6	FAISS	Facebook AI Similarity Search	Thư viện được phát triển bởi Facebook AI Research, được tối ưu hóa đặc biệt cho việc tìm kiếm tương tự trên quy mô lớn.
7	CNN	Convolutional Neural Networks	Mô hình mạng nơ ron tích chập, mô hình này đã cách mạng hóa lĩnh vực thị giác máy tính và xử lý ảnh kể từ khi được giới thiệu
8	ResNet	Residual Networks	Mô hình mạng CNN cho phép training các mạng với độ sâu lên đến 152-layer và đạt được performance vượt trội trên nhiều benchmark quan trọng.

DANH MỤC HÌNH ẢNH

Hình 1.1: Mô hình Content – Based Image Retrieval.....	10
Hình 1.2: Minh họa các thành phần chính của hệ thống CBIR.....	12
Hình 1.3: Quy trình xử lý truy vấn CBIR.....	13
Hình 1.4: Biểu diễn ảnh trong các không gian màu khác nhau.....	17
Hình 1.5: Quy trình SIFT.....	19
Hình 1.6: Cách thức so sánh của phương thức LBP.....	20
Hình 1.7: Kiến trúc tổng quan của CNN.....	26
Hình 1.8: Timeline phát triển các kiến trúc CNN từ 1998-2019.....	27
Hình 1.9: Minh họa dạng basic block và bottleneck block.....	30
Hình 1.10: Phương pháp Pre-trained model và Fine-tuning.....	35
Hình 1.11: Biểu đồ so sánh các dạng chuẩn hóa khác nhau.....	37
Hình 1.12: Minh họa việc áp dụng cosine similarity trong không gian.....	40
Hình 1.13: So sánh đa tầng (layer fusion, coarse - to - fine).....	44
Hình 2.1: Tích hợp service extraction, index Faiss và metadata.....	60

LỜI NÓI ĐẦU

1. Lý do chọn đề tài

Cùng với sự phát triển của công nghệ thông tin và Internet đã làm gia tăng nhanh chóng dữ liệu đa phương tiện khiến cho nhu cầu chia sẻ và tìm kiếm những loại tài nguyên này cũng được tăng theo, trong đó có tài nguyên dữ liệu kiểu hình ảnh. Tìm kiếm dữ liệu hình được sử dụng với nhiều mục đích khác nhau và được ứng dụng trong nhiều lĩnh vực quan trọng của cuộc sống như trong hệ thống bảo mật, an ninh, forensic hay các hệ thống thương mại. Vì thế việc có thêm những kiến thức về các thuật toán trong xây dựng hệ thống tìm kiếm ảnh ngày càng trở nên quan trọng đối với lập trình viên.

Cùng với những phương pháp tìm kiếm ảnh dựa trên từ khóa, tìm kiếm ảnh theo nội dung (CBIR – Content Based Image Retrieval), là một phương pháp mới, hiệu quả để tìm ra những kết quả hình ảnh tương tự dựa trên đặc trưng như màu sắc, hình dạng, kết cấu... mà không cần đến các thông tin mô tả bằng từ khóa. Việc nghiên cứu chuyên sâu về kỹ thuật này sẽ giúp chúng em hiểu rõ cơ chế của các hệ thống nhận dạng hình ảnh, đồng thời góp phần nâng cao độ chính xác và hiệu quả trong các ứng dụng phân tích và xử lý ảnh động.

2. Mục tiêu thực hiện đề tài

Nghiên cứu, tìm hiểu và xây dựng hệ thống web với chương trình truy vấn sản phẩm theo hình ảnh nhằm cho phép người dùng tìm kiếm thông tin sản phẩm một cách nhanh chóng và chính xác. Thay vì phải thực hiện nhập các từ khóa mô tả thì người dùng chỉ cần tải lên một bức ảnh, song hệ thống sẽ tự động trích xuất đặc trưng và trả về danh sách các sản phẩm tương đồng trong cơ sở dữ liệu.

Tìm hiểu và áp dụng các kỹ thuật, thuật toán trong xử lý ảnh và trích xuất đặc trưng của ảnh phục vụ cho truy vấn.

Xây dựng giao diện web đơn giản, thân thiện với người dùng để tìm kiếm kết quả mong muốn.

Triển khai hệ thống backend có khả năng tiếp nhận ảnh, xử lý ảnh và tìm kiếm các sản phẩm tương tự.

Tối ưu hóa thời gian phản hồi và độ chính xác của kết quả truy vấn, đồng thời đảm bảo khả năng mở rộng khi số dữ liệu tăng lên.

Thực hiện kiểm thử và đánh giá hiệu quả của hệ thống trong môi trường thực tế, đồng thời điều chỉnh và cải thiện tính năng dựa trên phản hồi của người dùng.

CHƯƠNG 1. GIỚI THIỆU VỀ CÔNG NGHỆ TÌM KIẾM ẢNH TƯƠNG TỰ

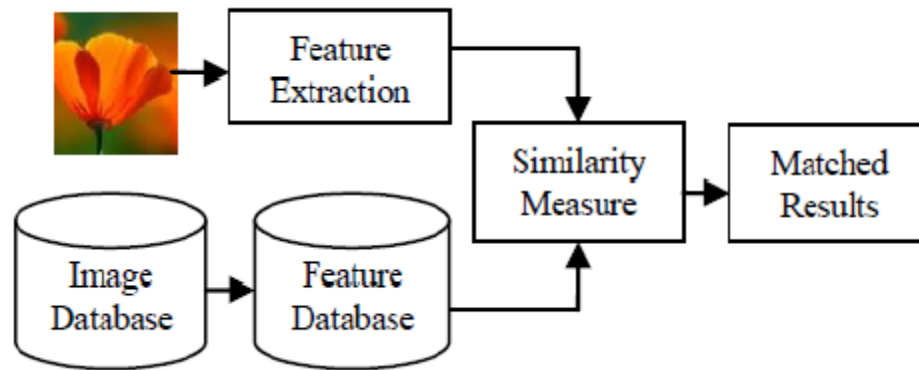
1.1. Tổng quan trong xử lý ảnh

1.1.1. Giới thiệu về kỹ thuật Content – Based Image Retrieval (CBIR)

1.1.1.1. Khái niệm

Content-Based Image Retrieval (CBIR), còn được gọi là Query by Image Content (QBIC) hoặc Query by Visual Content (QBVC), là một phương pháp tiếp cận trong lĩnh vực thị giác máy tính và truy xuất thông tin, nhằm tìm kiếm và truy xuất các hình ảnh từ cơ sở dữ liệu lớn dựa trên nội dung thị giác của chính hình ảnh đó. Khái niệm "content-based" nhấn mạnh rằng quá trình tìm kiếm được thực hiện dựa trên các đặc trưng thị giác (visual features) có thể được trích xuất tự động từ nội dung của hình ảnh, bao gồm màu sắc (color), kết cấu bề mặt (texture), hình dạng (shape), và bố cục không gian (spatial layout), thay vì dựa vào các thông tin metadata như từ khóa, nhãn, hoặc mô tả văn bản được gán thủ công.

CBIR được phân biệt rõ ràng với Text-Based Image Retrieval (TBIR) - phương pháp tìm kiếm hình ảnh truyền thống dựa trên từ khóa và metadata. Trong khi TBIR đòi hỏi việc chú thích thủ công cho mỗi hình ảnh thông qua các từ khóa hoặc mô tả văn bản, CBIR có khả năng phân tích trực tiếp nội dung pixel của hình ảnh để tự động trích xuất các đặc trưng đại diện. Sự khác biệt này không chỉ giảm đáng kể chi phí lao động mà còn loại bỏ tính chủ quan trong quá trình gán nhãn, đồng thời cho phép khám phá các mối tương quan thị giác mà con người có thể bỏ qua.



Hình 1.1: Mô hình Content – Based Image Retrieval

Từ góc độ toán học, CBIR có thể được mô hình hóa như một bài toán tối ưu trong không gian đa chiều. Giả sử có một cơ sở dữ liệu hình ảnh $D = \{I_1, I_2, \dots, I_n\}$ và một ảnh truy vấn Q , mục tiêu của CBIR là tìm một tập con $S \subseteq D$ sao cho:

$$S = \{I_i \in D \mid \text{sim}(f(Q), f(I_i)) > \text{threshold}\}$$

Trong đó $f(\cdot)$ là hàm trích xuất đặc trưng, và $\text{sim}(\cdot, \cdot)$ là hàm đo độ tương tự. Việc lựa chọn hàm f và sim phù hợp là trọng tâm của các nghiên cứu về CBIR.

1.1.1.2. Lịch sử hình thành và phát triển

Lịch sử phát triển của CBIR bắt đầu từ đầu thập niên 1990, khi lượng dữ liệu hình ảnh số bắt đầu gia tăng nhanh chóng cùng với sự phát triển của công nghệ máy tính và Internet. Động lực chính thúc đẩy sự ra đời của CBIR xuất phát từ nhận thức về những hạn chế căn bản của phương pháp tìm kiếm hình ảnh dựa trên văn bản truyền thống.

Hệ thống CBIR đầu tiên có ý nghĩa thương mại là QBIC (Query by Image Content), được phát triển bởi IBM vào năm 1993. QBIC đã giới thiệu nhiều khái

niệm tiên phong như truy vấn theo màu sắc, texture, và hình dạng, đặt nền móng cho toàn bộ lĩnh vực nghiên cứu sau này.

Thập niên tiếp theo chứng kiến sự bùng nổ của nghiên cứu CBIR với hàng loạt hệ thống được phát triển: Virage (1995), Excalibur (1996), NETRA của UC Santa Barbara (1997), và SIMPLIcity của Stanford (2000). Giai đoạn này được đánh dấu bởi sự đa dạng hóa trong các phương pháp trích xuất đặc trưng và thuật toán so sánh độ tương tự.

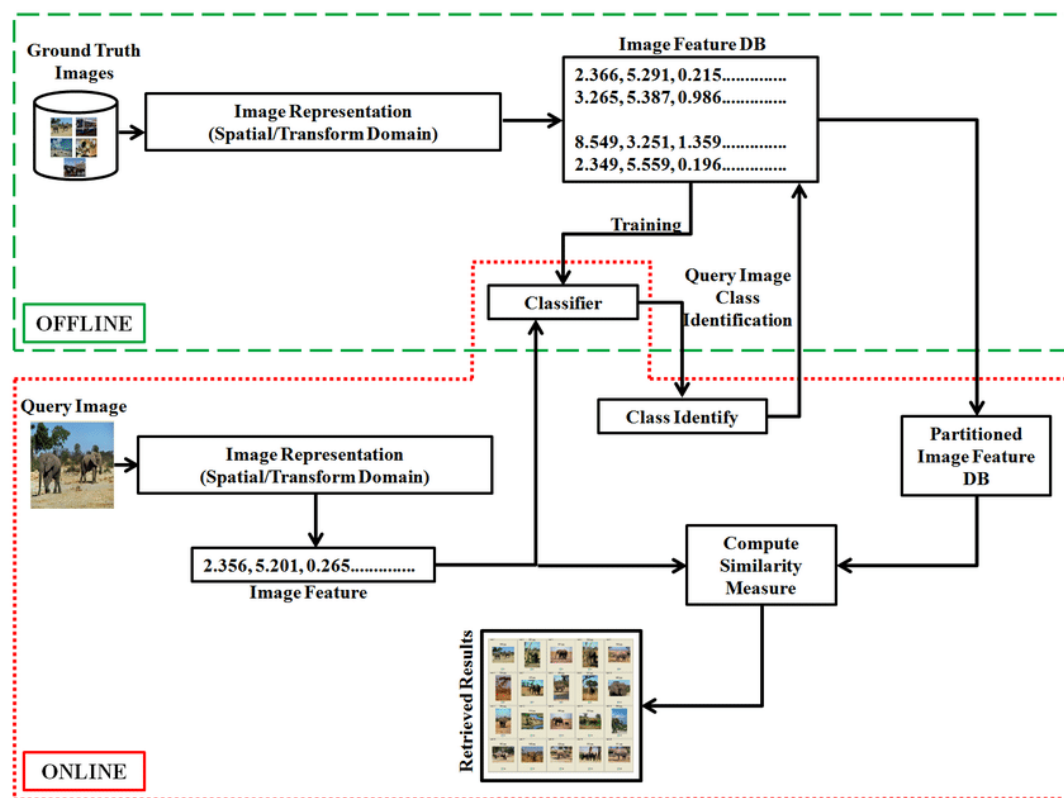
Một mốc quan trọng là việc Manjunath và Ma (1996) giới thiệu các bộ lọc Gabor cho việc phân tích texture, trong khi Swain và Ballard (1991) đã thiết lập phương pháp histogram màu như một đặc trưng cơ bản. Những đóng góp này tạo ra nền tảng lý thuyết vững chắc cho các nghiên cứu tiếp theo.

Vào giai đoạn những năm 2005 tới 2012, việc ứng dụng machine learning vào CBIR trở nên phổ biến với các kỹ thuật như Support Vector Machine (SVM), Random Forest, và các thuật toán clustering được sử dụng để cải thiện độ chính xác tìm kiếm.

Cuộc cách mạng Deep Learning bắt đầu với AlexNet (2012) đã thay đổi hoàn toàn bộ mặt của CBIR. Các mạng nơ-ron tích chập (CNN) như VGGNet, ResNet, và các kiến trúc hiện đại khác đã cho phép trích xuất các đặc trưng phức tạp và có tính ngữ nghĩa cao, giải quyết một phần đáng kể vấn đề "semantic gap" - khoảng cách giữa đặc trưng thị giác mức thấp và ý nghĩa ngữ nghĩa mức cao.

1.1.1.3. Nguyên lý hoạt động cơ bản

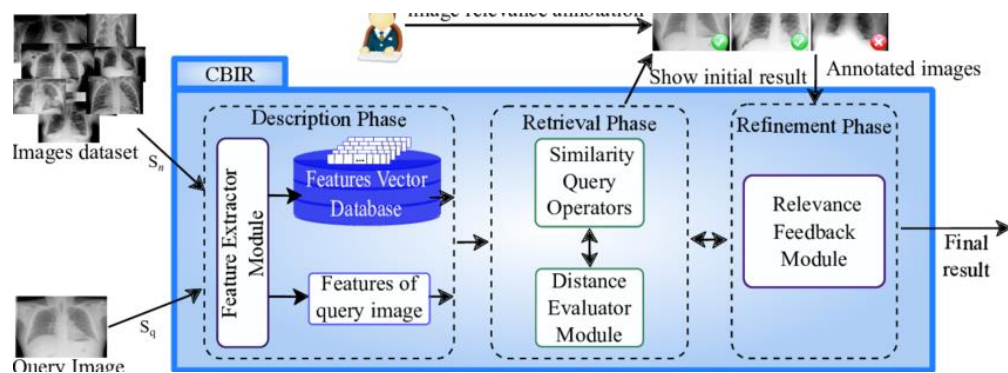
Hệ thống CBIR hoạt động theo một pipeline phức tạp bao gồm nhiều giai đoạn xử lý tuần tự và song song. Nguyên lý cốt lõi là chuyển đổi thông tin thị giác định tính thành các biểu diễn số học định lượng, từ đó cho phép áp dụng các phép toán toán học để so sánh và đánh giá độ tương tự.



Hình 1.2: Minh họa các thành phần chính của hệ thống CBIR

Nguyên lý hoạt động của hệ thống CBIR được chia làm 4 giai đoạn:

- **Tiền xử lý ảnh:** Giai đoạn tiền xử lý đóng vai trò quan trọng trong việc chuẩn hóa dữ liệu đầu vào và cải thiện chất lượng các đặc trưng được trích xuất bao gồm chuẩn hóa về kích thước, màu sắc và độ sáng.
- **Trích xuất đặc trưng:** Đây là giai đoạn quan trọng thông tin thị giác phong phú của hình ảnh được chuyển đổi thành các vector số học có thể xử lý được. Các đặc trưng được chia thành ba loại chính: đặc trưng về màu sắc, đặc trưng về kết cấu và đặc trưng về hình dạng.
- **Lập chỉ mục và lưu trữ:** Với khối lượng dữ liệu lớn, việc tìm kiếm tuyến tính sẽ không khả thi. Do đó, các cấu trúc chỉ mục hiệu quả là cần thiết.
- **Truy vấn và so sánh:** Giai đoạn cuối cùng đó chính là việc xử lý ảnh truy vấn và tìm các ảnh tương tự đang có sẵn trong cơ sở dữ liệu.



Hình 1.3: Quy trình xử lý truy vấn CBIR

1.1.2. Các công đoạn trong xử lý ảnh

Xử lý ảnh trong hệ thống CBIR là một quy trình phức tạp bao gồm nhiều giai đoạn khác nhau, mỗi giai đoạn đều đóng vai trò quan trọng trong việc đảm bảo chất lượng và hiệu quả của việc tìm kiếm ảnh. Quy trình này thường được chia làm 3 giai đoạn chính đó là tiền xử lý ảnh (preprocessing), trích xuất đặc trưng (feature extraction), và tìm kiếm thông qua độ tương đồng (similarity matching). Mỗi giai đoạn đều có những thách thức và yêu cầu kỹ thuật riêng biệt, đồng thời chúng tương tác với nhau để tạo ra một hệ thống tìm kiếm ảnh hiệu quả.

1.1.2.1. Tiền xử lý ảnh (Image Preprocessing)

Tiền xử lý ảnh là giai đoạn đầu tiên và quan trọng nhất trong quá trình xử lý ảnh, đóng vai trò như một bước chuẩn bị cần thiết trước khi thực hiện trích xuất đặc trưng. Tầm quan trọng của việc tiền xử lý dữ liệu được thể hiện qua việc:

- **Chuẩn hóa dữ liệu đầu vào:** Đảm bảo tất cả ảnh trong cơ sở dữ liệu có format và kích thước nhất quán.
- **Giảm thiểu ảnh hưởng của nhiễu:** Loại bỏ các yếu tố gây nhiễu, thứ có thể làm sai lệch kết quả khi trích xuất đặc trưng ảnh.
- **Tăng cường thông tin hữu ích:** Làm nổi bật các đặc trưng quan trọng của ảnh.
- **Tối ưu hóa tài nguyên tính toán:** Giảm kích thước dữ liệu mà giữ được nhưng đặc trưng cơ bản và thông tin quan trọng.

1.1.2.1.1. Các kỹ thuật tiền xử lý cơ bản

Resize (Thay đổi kích thước) là một trong những bước tiền xử lý cơ bản nhất, nhằm mục đích chuẩn hóa kích thước của tất cả ảnh trong hệ thống. Việc này đặc biệt quan trọng khi làm việc với các mô hình học sâu, vì chúng thường yêu cầu đầu vào có kích thước cố định.

Một vài phương pháp Resize phổ biến gồm có:

- **Nearest Neighbor Interpolation:** Phương thức đơn giản nhất, sử dụng giá trị pixel gần nhất.
- **Bilinear Interpolation:** Sử dụng 4-pixel lân cận để tính toán giá trị mới.
- **Bicubic Interpolation:** Sử dụng 16-pixel lân cận, cho kết quả mượt mà hơn nhưng lại có độ tính toán phức tạp cao.

Normalization (Chuẩn hóa) đảm bảo các giá trị pixel nằm trong một khoảng nhất định, thường là $[0,1]$ hoặc $[-1,1]$. Công thức chuẩn hóa thường dùng:

$$I_{\text{normalized}} = (I - I_{\text{min}}) / (I_{\text{max}} - I_{\text{min}})$$

Với I là ảnh gốc, I_{min} và I_{max} là giá trị minimum và maximum của ảnh.

Noise Reduction (Giảm nhiễu) là quá trình loại bỏ các thông tin không mong muốn trong ảnh, có thể xuất hiện do điều kiện chụp ảnh, thiết bị, hoặc quá trình truyền tải.

Các kỹ thuật lọc nhiễu phổ biến:

- **Gaussian Filter:** Sử dụng kernel Gaussian để làm mờ ảnh.

- **Median Filter:** Thay thế giá trị pixel bằng giá trị trung vị của các pixel lân cận, hiệu quả trong việc khử nhiễu salt and pepper noise.
- **Bilateral Filter:** Giữ nguyên edges trong khi làm mờ vùng đồng nhất.

Histogram Equalization là kỹ thuật tăng cường độ tương phản của ảnh bằng cách phân phối lại các mức xám sao cho histogram trở nên đều hơn. Điều này giúp cải thiện chất lượng ảnh, đặc biệt với những ảnh có độ tương phản thấp.

Công thức histogram equalization:

$$s = T(r) = (L-1) * \sum_{k=0}^r P_r(k)$$

Trong đó:

s: giá trị pixel sau khi cân bằng

r: giá trị pixel gốc

L: số mức xám (thường là 256)

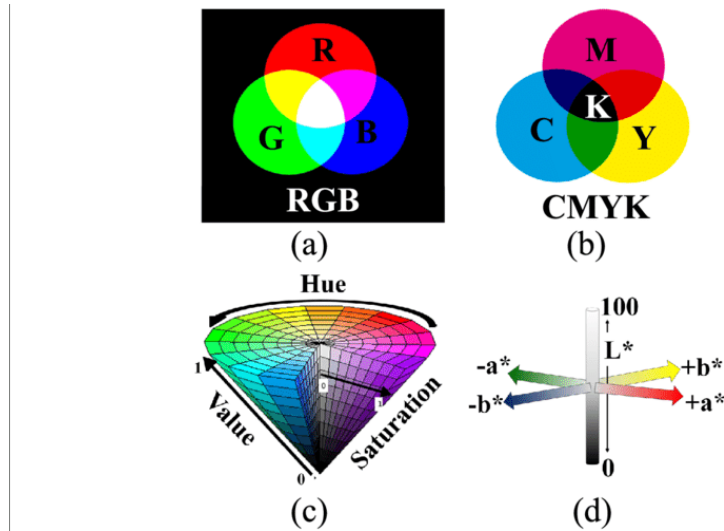
$P_r(k)$: xác suất của mức xám k

Color Space Conversion là quá trình chuyển đổi ảnh từ không gian màu này sang không gian màu khác, tùy thuộc vào mục đích sử dụng cụ thể.

Các không gian màu phổ biến:

- **RGB (Red, Green, Blue):** Không gian màu cơ bản, phù hợp với thiết bị hiển thị.
- **HSV (Hue, Saturation, Value):** Tách biệt thông tin màu sắc và độ sáng.

- **LAB:** Không gian màu độc lập với thiết bị, phù hợp cho so sánh màu sắc.
- **YUV:** Tách thông tin độ sáng (luminance) và màu (chrominance).



Hình 1.4: Biểu diễn ảnh trong các không gian màu khác nhau

1.1.2.2. Trích xuất đặc trưng (Feature Extraction)

1.1.2.2.1. Khái niệm về đặc trưng ảnh

Đặc trưng ảnh (image features) là những thông tin đặc biệt được trích xuất từ ảnh, đại diện cho nội dung và tính chất của ảnh một cách compact và có ý nghĩa. Đặc trưng tốt phải thỏa mãn các yêu cầu: tính phân biệt (discriminative), tính bất biến (invariant) với các phép biến đổi như xoay, dịch chuyển, thay đổi kích thước, và tính nhiễu.

1.1.2.2.2. Phân loại đặc trưng

Low- level features là những đặc trưng cơ bản được trích xuất trực tiếp từ dữ liệu pixel, không cần kiến thức về ngữ nghĩa của ảnh.

- **Đặc trưng màu sắc (Color Features):** Màu sắc là một trong những đặc trưng trực quan nhất của ảnh. Các phương pháp trích xuất đặc trưng màu sắc phổ biến có thể kể đến là Color Histogram giúp phân phối thống kê của các màu trong ảnh, Color Moments, Color Coherence Vector.
- **Đặc trưng texture (Texture Features):** Texture mô tả tính chất bề mặt và cấu trúc không gian của ảnh.
- **Đặc trưng hình dạng (Shape Features):** Hình dạng mô tả contour (khô, đường viền, đường nét) và cấu trúc hình học của đối tượng.

Mid- level features kết hợp thông tin từ nhiều low- level features để tạo ra một sự biểu diễn phức tạp hơn.

High- level features chứa thông tin ngữ nghĩa về nội dung hình ảnh, thường được trích xuất bởi các mô hình học sâu.

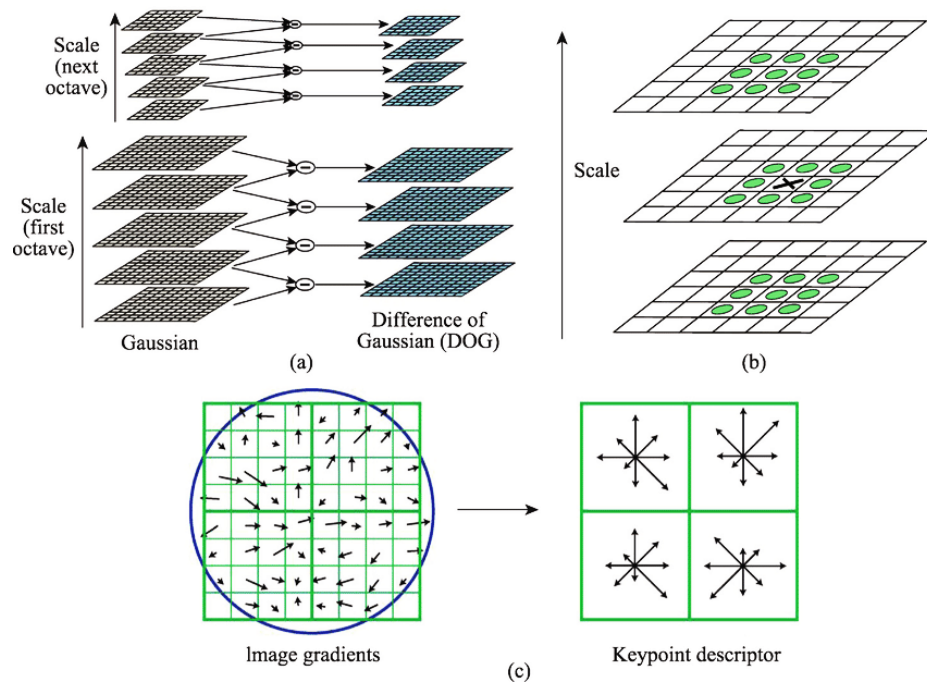
1.1.2.2.3. Các phương pháp trích xuất đặc trưng truyền thống

SIFT (Scale – Invariant Feature Transform) là một trong những descriptor mạnh mẽ trong việc so sánh ảnh, được phát triển bởi David Lowe (2004). SIFT có khả năng bất biến với scale, rotation, robust và illumination changes.

Quy trình SIFT:

- Scale – space extrema detection: Tìm đặc trưng ứng viên.

- Keypoint localization: Loại bỏ keypoint có độ tương phản thấp và edge response cao.
- Orientation assignment: Gán hướng chính cho mỗi keypoint.
- Keypoint descriptor: Tạo descriptor 128 chiều cho mỗi keypoint.



Hình 1.5: Quy trình SIFT

SURF (Speeded Up Robust Features) được phát triển như một phiên bản tối ưu hơn của SIFT, nhanh hơn khoảng 3 – 5 lần nhưng vẫn đảm bảo được performance tương đương.

Một vài đặc điểm cải tiến của SURF:

- Hessian matrix – based detector: Sử dụng định thức của ma trận Hessian.

- Integral images: Tăng tốc độ tính toán.
- Wavelet – based descriptor: Sử dụng Haar wavelet response.

HOG (Histogram of Oriented Gradients) đặc biệt hiệu quả trong phát hiện vật thể và con người, ứng dụng trong phát hiện người và nhận dạng vật thể.

Quy trình của HOG:

- Gradient computation: Tính gradient magnitude và direction.
- Orientation binning: Chia góc thành 9 bins (0° - 180°).
- Descriptor blocks: Tạo histogram cho mỗi cell (8x8 pixel).
- Block normalization: Chuẩn hóa histogram trong mỗi block (2x2 cells).

LBP (Local Binary Patterns) là một descriptor texture đơn giản nhưng hiệu quả, LBP với các biến thể bao gồm: Uniform LBP – xét các pattern có tối đa 2 transition, Rotation Invariant LBP – bất biến với rotation, Multi – scale LBP – sử dụng nhiều radius khác nhau.

'00001101' = 13	'00011010' = 26	'00110100' = 52	'01101000' = 104
'11010000' = 208	'10100001' = 161	'01000011' = 67	'10000110' = 134

Hình 1.6: Cách thức so sánh của phương thức LBP

1.1.2.3. Tìm kiếm và so sánh (Similarity Matching)

Sau khi trích xuất được đặc trưng, bước tiếp theo là tìm kiếm những ảnh có đặc trưng tương tự với ảnh truy vấn. Đây là giai đoạn quyết định hiệu suất của toàn bộ hệ thống CBIR. Dưới đây là một vài thuật toán tìm kiếm.

1.1.2.3.1. Linear Search

Linear Search là phương pháp tìm kiếm đơn giản nhất, so sánh ảnh truy vấn với từng ảnh trong cơ sở dữ liệu.

Thuật toán:

Function linearSearch (query, database):

```
results = []
```

```
for each image in database:
```

```
    similarity = computeSimilarity (query, image)
```

```
    results.append (image, similarity)
```

```
return sort (results, by=similarity, descending=True)
```

Ưu điểm:

- Đơn giản, dễ implement.
- Đảm bảo tìm được kết quả gần chính xác nhất.
- Không cần cấu trúc dữ liệu phức tạp.

Nhược điểm:

- Độ phức tạp $O(n)$, chậm với database lớn.

1.1.2.3.2. Tree - based Search

Tree – based Search sử dụng cấu trúc cây để tổ chức dữ liệu, giúp giảm thời gian tìm kiếm xuống $O(\log n)$. Một vài thuật toán biến thể của thuật toán này:

R – Tree được thiết kế đặc biệt cho dữ liệu không gian đa chiều. Cấu trúc của thuật toán được xây dựng nên bởi mỗi node chứa MBR (Minimum Bounding Rectangle), Leaf nodes chứa các data objects và internal nodes chứa các child nodes.

Thuật toán tìm kiếm:

function RTreeSearch (query, node):

 if node is leaf:

 return exactMatch (query, node. objects)

 else:

 candidates = []

 for child in node. children:

 if overlap (query.MBR, child.MBR):

 candidates.extend(RTreeSearch(query,child))

 return candidates

K – d Tree (K – dimensional tree) là cấu trúc cây nhị phân để tổ chức điểm trong không gian k chiều. Cấu trúc của thuật này được tạo nên bởi mỗi node chia không gian theo một dimension, các rotation qua dimensions và balanced tree với độ sâu $O(\log n)$.

Thuật toán tìm kiếm:

```
function buildKDTree (points, depth=0):  
  
    if points are empty:  
  
        return null;  
  
    dim = depth % k  
  
    points.sort(by=dim)  
  
    median = len(points) // 2  
  
    node.value = points[median]  
  
    node.left = buildKDTree(points[:median], depth+1)  
  
    node.right = buildKDTree(points[median+1:], depth+1)  
  
    return node
```

1.1.2.3.3. Approximate Nearest Neighbor (Tìm kiếm ảnh tương đồng)

FAISS (Facebook AI Similarity Search) là thư viện được phát triển bởi Facebook AI Research, được tối ưu hóa đặc biệt cho việc tìm kiếm tương tự trên quy mô lớn (large – scale similarity search) và hỗ trợ tăng tốc GPU. Trong hệ thống CBIR được phát triển, FAISS được sử dụng làm công cụ chính cho việc tìm kiếm ảnh tương tự.

Các loại index trong FAISS:

- Flat index: Phương pháp tìm kiếm brute-force, so sánh trực tiếp với tất cả vectors trong database. Đảm bảo kết quả chính xác nhất và có độ phức tạp $O(n)$.

- IVF (Inverted File Index): Chia không gian đặc trưng thành các cluster, chỉ tìm kiếm trong các clusters gần nhất với query. Giảm độ phức tạp xuống $O(\sqrt{n})$ với một chút trade-off về độ chính xác.
- PQ (Produce Quantization): Nén vectors thành các codes ngắn hơn, tiết kiệm memory đáng kể.
- HNSW (Hierarchical Navigable Small World): Xây dựng các graph đa tầng để tìm kiếm nhanh với độ chính xác cao.

1.1.2.4. Kết luận

Quy trình xử lý ảnh trong CBIR bao gồm ba giai đoạn chính, mỗi giai đoạn đều có vai trò quan trọng và ảnh hưởng trực tiếp đến hiệu suất tổng thể của hệ thống. Tiền xử lý ảnh đảm bảo chất lượng dữ liệu đầu vào, trích xuất đặc trưng tạo ra sự khác biệt và thể hiện nhỏ gọn trong khi tìm kiếm và so sánh đảm bảo hiệu quả và chính xác trong việc matching. Sự phát triển của các kỹ thuật học sâu đã mang lại nhiều cải tiến đáng kể, nhưng các phương pháp truyền thống vẫn có giá trị trong nhiều ứng dụng cụ thể. Việc lựa chọn phương pháp phù hợp cần cân nhắc tới các yêu cầu như độ chính xác yêu cầu, tài nguyên tính toán, kích thước cơ sở dữ liệu, và yêu cầu về thời gian thực.

1.1.3. Mô hình học sâu (CNN) – ResNet

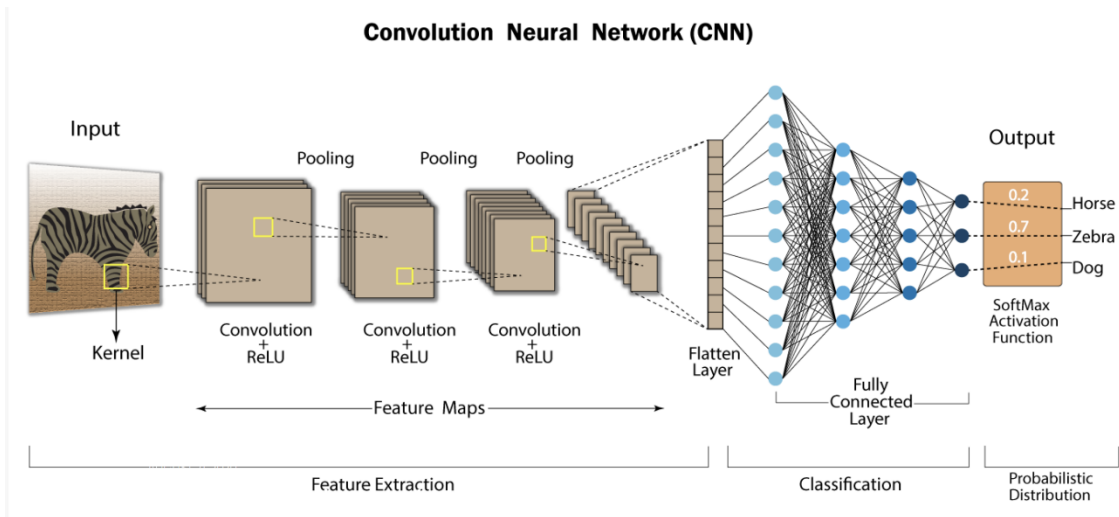
1.1.3.1. Giới thiệu

CNN – Convolutional Neural Networks hay còn được gọi là mô hình mạng nơ ron tích chập, mô hình này đã cách mạng hóa lĩnh vực thị giác máy tính và xử lý ảnh kể từ khi được giới thiệu. CNN được thiết kế đặc biệt để xử lý dữ liệu có cấu trúc dạng lưới như hình ảnh, với khả năng tự động học và trích xuất các đặc trưng phân cấp từ dữ liệu đầu vào

Mạng nơ ron tích chập có nguồn gốc từ nghiên cứu về hệ thống thị giác của động vật. Năm 1980, Kunihiko Fukushima đã giới thiệu Neocognitron, một trong những mô hình đầu tiên sử dụng kiến trúc tương tự CNN. Tuy nhiên, bước đột phá thực sự đến vào năm 1998 khi Yann LeCun và cộng sự phát triển LeNet-5 để nhận dạng chữ số viết tay.

Sự phát triển vượt bậc của CNN bắt đầu từ năm 2012 với AlexNet của Alex Krizhevsky, đã giành chiến thắng trong cuộc thi ImageNet Large Scale Visual Recognition Challenge (ILSVRC) với độ chính xác vượt trội so với các phương pháp truyền thống. Từ đó, các kiến trúc CNN ngày càng được cải tiến với VGGNet (2014), GoogLeNet (2014), và đặc biệt là ResNet (2015) đã đánh dấu một bước ngoặt quan trọng trong việc xây dựng mạng nơ-ron sâu.

Hệ thống kiến trúc cơ bản của CNN:



Hình 1.7: Kiến trúc tổng quan của CNN

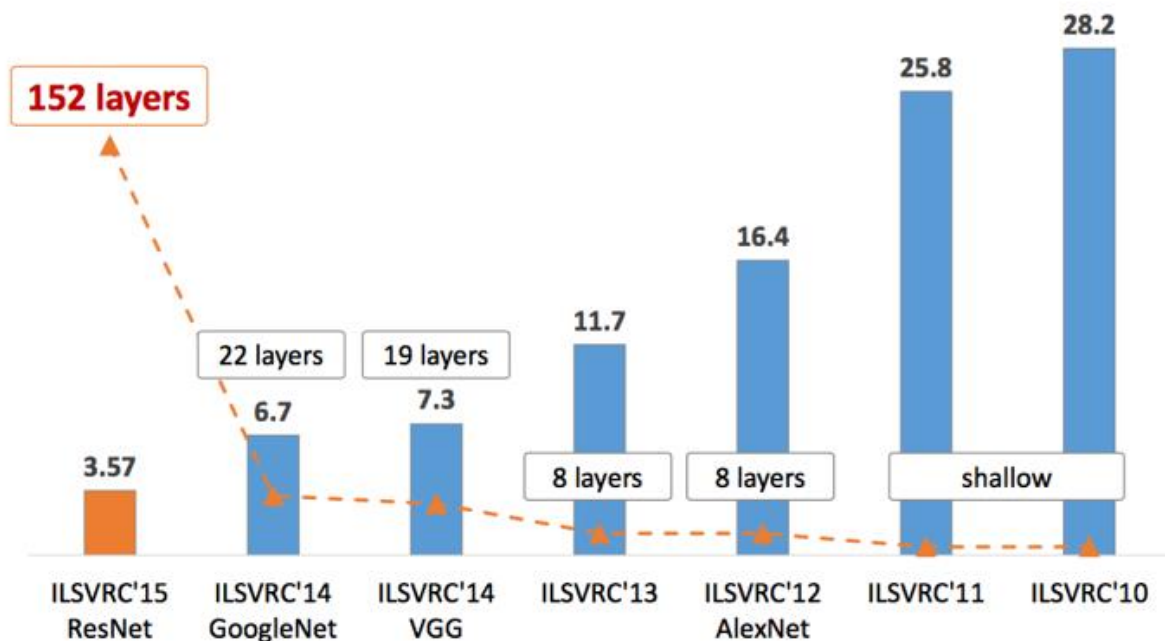
CNN bao gồm các thành phần chính sau:

- **Lớp tích chập (Convolutional Layer):** Đây là thành phần cốt lõi của CNN, thực hiện phép tích chập giữa đầu vào và một tập hợp các bộ lọc (filters/ kernels). Mỗi bộ lọc học để phát hiện một đặc trưng cụ thể như cạnh, góc, hoặc texture.
- **Lớp gộp (Pooling Layer):** Lớp này giảm kích thước không gian của features maps, giúp giảm số tham số và computational cost. Hai loại pooling phổ biến nhất là Max Pooling và Average Pooling.
- **Lớp kết nối đầy đủ (Fully Connected Layer):** Thường được đặt ở cuối mạng, lớp này kết nối tất cả các nơ ron từ lớp trước và thực hiện phân loại cuối cùng.
- **Hàm kích hoạt (Activation Functions):** Hàm được sử dụng rộng rãi nhất là ReLU (Rectified Linear Unit) - $f(x) = \max(0, x)$.

Ưu điểm của CNN trong xử lý ảnh:

- Translation Invariance: CNN có khả năng nhận dạng đối tượng bất kể vị trí của chúng.
- Local Connectivity: Mỗi nơon chỉ kết nối với một vùng nhỏ của đầu vào, giúp giảm số tham số và tận dụng tính chất cục bộ của ảnh.
- Hierarchical Feature Learning: CNN tự động học các đặc trưng từ low-level (cạnh, góc) đến high-level (đối tượng phức tạp) qua các lớp.
- Parameter Sharing: Cùng một bộ lọc được sử dụng trên toàn bộ ảnh, giúp giảm đáng kể số tham số cần học.

1.1.3.2. Các mô hình CNN điển hình



Hình 1.8: Timeline phát triển các kiến trúc CNN từ 1998-2019

LeNet-5 (1998): Được thiết kế bởi Yann LeCun, LeNet-5 là một trong những CNN đầu tiên được áp dụng thành công. Kiến trúc gồm 2 lớp tích chập, 2 lớp pooling, và 3 lớp fully connected. Mặc dù đơn giản, LeNet-5 đã đặt nền móng cho các kiến trúc CNN hiện đại.

AlexNet (2012): AlexNet đánh dấu sự trở lại của deep learning trong computer vision. Với 8 lớp (5 convolutional và 3 fully connected), AlexNet đã giành chiến thắng ILSVRC 2012 với top-5 error rate 15.3%. Các đóng góp chính bao gồm:

- Sử dụng ReLU activation thay vì sigmoid/ tanh.
- Dropout để giảm overfitting.
- Data augmentation techniques.
- GPU acceleration.

VGGNet (2014): VGGNet của nhóm Visual Geometry Group (Oxford) đã chứng minh rằng độ sâu của mạng là yếu tố quan trọng. VGG-16 và VGG-19 sử dụng các convolutional filters nhỏ (3×3) nhưng có nhiều lớp hơn, đạt hiệu suất tốt hơn AlexNet.

GoogLeNet/Inception (2014): GoogLeNet giới thiệu Inception module, cho phép mạng học ở nhiều scale khác nhau trong cùng một lớp. Với 22 lớp nhưng chỉ 4 triệu tham số (ít hơn AlexNet 15 lần), GoogLeNet đã đạt top-5 error rate 6.67%.

So sánh hiệu suất và đặc điểm:

Mô hình	Năm	Layers	Parameters	Đặc điểm chính
LeNet – 5	1998	7	60K	CNN đầu tiên thành công
AlexNet	2012	8	60M	ReLu, Dropout, GPU
VGGNet – 16	2014	16	138M	Small filters, Deep network
GoogleNet	2014	22	4M	Inception modules
ResNet – 50	2015	50	25.6M	Residual connections

1.1.3.3. Mô hình ResNet

1.1.3.3.1. Giới thiệu

Residual Networks (ResNet), được giới thiệu bởi He et al. (2015), đã cách mạng hóa việc training các mạng nơ-ron sâu bằng cách giải quyết hiệu quả vấn đề vanishing gradient - một trong những thách thức lớn nhất trong deep learning. ResNet không chỉ cho phép training các mạng với độ sâu lên đến 152 layers mà còn đạt được performance vượt trội trên nhiều benchmark quan trọng.

1.1.3.3.2. Kiến trúc cơ bản của ResNet

Residual Block – Building Block thực hiện phép biến đổi:

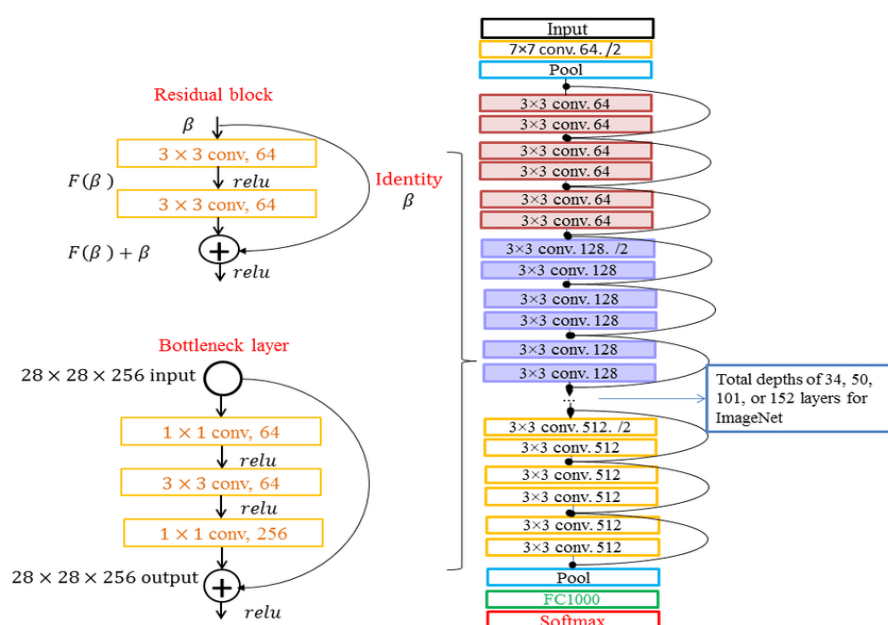
$$Y = F(x, \{W_i\}) + x$$

Trong đó:

- $F(x, \{W_i\})$: ánh xạ residual block được học.
- x : kết nối phím tắt nhận dạng
- y : đầu ra của khối

Bottleneck Design – thiết kế giúp tối ưu hóa hiệu quả tính toán được thiết kế đặc biệt cho ResNet – 50/ 101/ 152:

1. 1x1 convolution: giảm dimension.
2. 3x3 convolution: trích xuất đặc trưng.
3. 1x1 convolution: phục hồi dimension.



Hình 1.9: Minh họa dạng basic block và bottleneck block

Một vài thiết kế của mô hình ResNet có thể kể:

- ResNet – 18/34 sử dụng core là basic residual blocks
 - ResNet- 18: 18 layers, 11.7M parameters.
 - ResNet- 34: 34 layers, 21.8M parameter.
- ResNet – 50/101/152 sử dụng core là bottleneck design
 - ResNet- 50: 50 layers, 25.6M parameters.
 - ResNet- 101: 101 layers, 44.5M parameters.
 - ResNet- 152: 152 layers, 60.2M parameters.

Một vài thiết kế nâng cao hơn có thể kể đến là ResNeXt – sử dụng kết hợp giữa residual learning với grouped convolutions, tăng hiệu quả trong trích xuất dạng đặc trưng, hay Wide ResNet – tối ưu chiều rộng và chiều sâu giúp đạt hiệu suất cao hơn với ít lớp hơn.

1.1.3.3.3. Ứng dụng của ResNet trong CBIR

Trong hệ thống CBIR, ResNet đóng vai quan trọng như một tính năng mạnh mẽ trong trích xuất đặc trưng:

Quá trình xử lý ảnh khi sử dụng ResNet:

1. Pre-trained Model Loading: Sử dụng model ResNet trên ImageNet.
2. Sửa đổi kiến trúc: Xóa bỏ lớp phân loại cuối cùng.
3. Trích xuất đặc trưng: Trích xuất đặc trưng từ lớp pooling average (2048 – Dimension cho ResNet- 50).
4. So sánh độ tương đồng: So sánh độ tương đồng dựa trên Cosine hoặc khoảng cách Euclid.

Ưu điểm khi sử dụng ResNet trong CBIR:

- Đặc trưng ngữ nghĩa: Nắm bắt thông tin ngữ nghĩa ở mức cao.
- Transfer Learning: Khả năng khái quát hóa từ bộ dữ liệu của ImageNet.
- Hiệu quả tính toán: Sử dụng các đặc trưng được trích xuất sẵn để đảm bảo truy xuất nhanh chóng.
- Tính ổn định: Duy trì hiệu suất ổn định trên nhiều loại hình ảnh khác nhau.

1.2. Mô hình trích xuất đặc trưng ảnh sử dụng mạng CNN

1.2.1. Cách trích xuất vector đặc trưng từ ảnh truy vấn

Sau khi đã giới thiệu các mô hình CNN phổ biến và kiến trúc ResNet trong CBIR, phần này sẽ đi sâu vào kỹ thuật Transfer Learning – một phương pháp quan trọng để tận dụng sức mạnh của các mô hình đã được huấn luyện trước nhằm trích xuất đặc trưng ảnh hiệu quả.

1.2.1.1. Transfer Learning trong CBIR

Transfer Learning đã trở thành một mô hình quan trọng trong lĩnh vực Computer Vision – Thị giác máy tính nói chung và Content – Based Image Retrieval (CBIR) nói riêng. Khái niệm cốt lõi của Transfer Learning là tận dụng kiến thức đã học được từ một tác vụ (source task) để cải thiện hiệu quả học trên một tác vụ khác có liên quan (target task). Trong bối cảnh CBIR, điều này có nghĩa là sử dụng mô hình học sâu CNN đã được pre – trained trên các tập dữ liệu lớn như ImageNet để trích xuất đặc trưng cho các ảnh trong miền ứng dụng cụ thể.

Việc áp dụng Transfer Learning mang lại nhiều lợi ích đáng kể cho các hệ thống CBIR. Đầu tiên, nó giúp giảm đáng kể thời gian huấn luyện một mô hình CNN từ đầu với hàng triệu tham số, chúng ta có thể sử dụng các mô hình đã được pre – trained và chỉ cần fine – tune hoặc sử dụng chúng như feature extractor. Thứ hai, Transfer Learning giúp khắc phục vấn đề thiếu dữ liệu huấn luyện, một thách thức phổ biến trong nhiều ứng dụng CBIR thực tế. Các mô hình pre – trained đã học được các đặc trưng tổng quát từ hàng triệu ảnh, do đó có thể tạo ra các biểu diễn đặc trưng có chất lượng ảnh cao ngay cả khi tập dữ liệu target có kích thước nhỏ.

Pre - trained Models phổ biến:

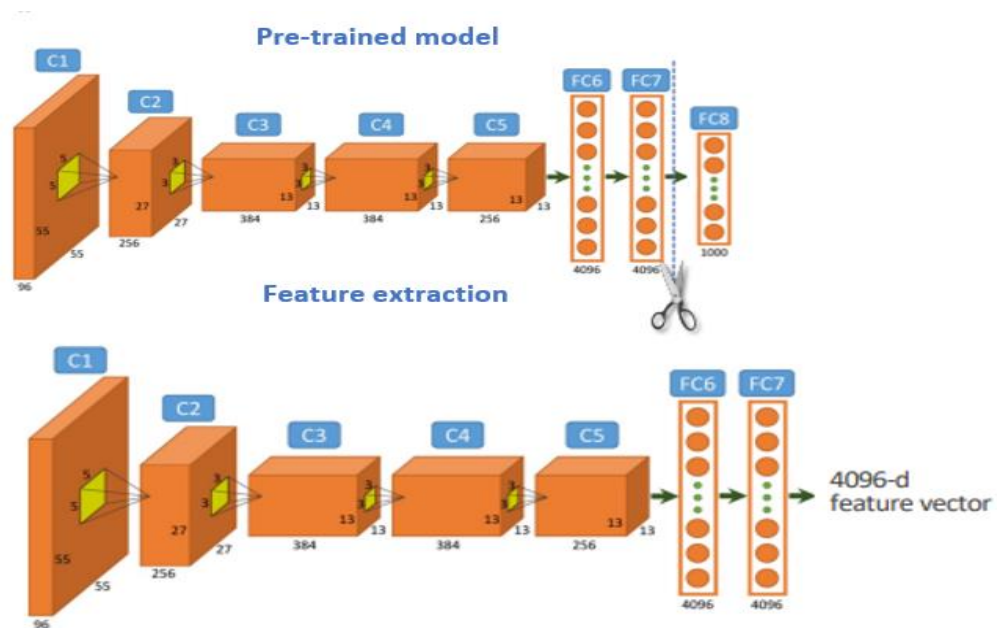
- **ImageNet Pretrained Models:** ImageNet là tập dữ liệu tiêu chuẩn với 14 triệu ảnh thuộc 20.000 danh mục khác nhau. Các mô hình được pre- trained trên ImageNet như ResNet, VGG, Inception, và DenseNet đã được chứng minh hiệu quả cao trong việc học các đặc trưng visual tổng quát. Những đặc trưng này có thể được transfer hiệu quả sang nhiều task khác nhau từ nhận diện vật thể tới phân loại chi tiết danh mục.
- **Domain – specific Pretrained Models:** Đối với các ứng dụng chuyên biệt, việc sử dụng các mô hình được pre – trained trên dữ liệu gần với domain target sẽ mang lại hiệu quả cao hơn. Ví dụ đối với hệ thống nhận diện ảnh trong y tế, các mô hình được pre- trained trên ChestX-ray datasets sẽ hiệu quả hơn so với ImageNet pretrained models.

Phương pháp fine – tuning phổ biến:

- **Feature Extraction Approach:** Trong phương pháp này, mô hình pre – trained được sử dụng như một feature extractor cố định. Tất cả các layer của mô hình được “đóng băng”, và chỉ có layer phân loại cuối cùng được thay thế và huấn luyện. Phương pháp này phù hợp khi tập dữ liệu target có kích thước nhỏ và domain tương tự với domain pre – trained.
- **Fine – tuning Approach:** Ở phương pháp này, toàn bộ hoặc một phần của mô hình pre – trained sẽ được tiếp tục huấn luyện trên dữ liệu target với learning rate thấp. Việc sử dụng learning rate thấp giúp

bảo tồn các đặc trưng đã học được từ pre – training đồng thời cho phép mô hình thích ứng với đặc điểm của dữ liệu mới.

- **Joint Training Approach:** Đây là phương pháp kết hợp giữa việc huấn luyện trên dữ liệu source và target đồng thời. Phương pháp này đặc biệt hiệu quả khi có quyền truy cập vào cả dữ liệu pre – training và target data.



Hình 1.10: Phương pháp Pre-trained model và Fine-tuning trong trích xuất đặc trưng

1.2.1.2. Quy trình trích xuất đặc trưng

1.2.1.2.1. Lựa chọn layer phù hợp để trích xuất đặc trưng features

Việc lựa chọn layer để trích xuất đặc trưng là một quyết định quan trọng, ảnh hưởng trực tiếp đến chất lượng của hệ thống CBIR. Trong kiến trúc CNN, các layer khác nhau sẽ học các mức độ trừu tượng khác nhau của đặc trưng. Các layer đầu (shallow layers) thường học các đặc trưng low-level như edges, corners, và textures. Ngược lại, các layer sâu hơn (deeper layers) học các đặc trưng high-level và mang tính học thuật (semantic) hơn.

Đối với CBIR, việc lựa chọn layer trích xuất phụ thuộc vào bản chất của tác vụ retrieval. Nếu mục tiêu là tìm kiếm các ảnh có nội dung semantic tương tự, các layer sâu hơn sẽ phù hợp. Ngược lại, nếu mục tiêu là tìm kiếm dựa trên texture hoặc pattern tương tự, các layer trung gian có thể hiệu quả hơn. Trong thực tế, layer trước các layer được kết nối cuối cùng (thường là global average pooling layer) thường được sử dụng vì nó cung cấp sự cân bằng tốt giữa tính semantic và tính phân loại.

1.2.1.2.2. Feature Vector Dimensions và Compression

Kích thước của feature vector là một yếu tố cần cân nhắc kỹ lưỡng. Các mô hình hiện đại như ResNet-152 có thể tạo ra feature vector với chiều cao (2048 dimensions), điều này có thể gây ra vấn đề về memory và tốc độ tính toán trong các ứng dụng large-scale. Việc giảm kích thước không gian (dimensionality reduction) trở thành cần thiết để cân bằng giữa chất lượng đặc trưng và hiệu quả tính toán.

Principal Component Analysis (PCA) là một phương pháp phổ biến để giảm kích thước của feature vector. PCA không chỉ giúp giảm kích thước storage mà

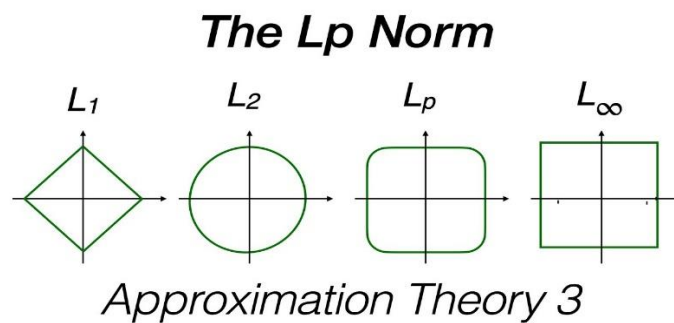
còn có thể loại bỏ noise và redundant information, từ đó cải thiện hiệu quả retrieval. Tuy nhiên, việc lựa chọn số chiều sau PCA cần được thực hiện cẩn thận để tránh mất mát thông tin quan trọng.

1.2.1.2.3. Kỹ thuật chuẩn hóa

L2 Normalization là kỹ thuật chuẩn hóa phổ biến nhất trong CBIR. L2 normalization đảm bảo rằng tất cả feature vectors có kích cỡ bằng 1, điều này giúp cosine similarity trở thành một thước đo đo lường hiệu quả.

PCA Dimensionality Reduction: Như đã đề cập, PCA không chỉ giúp giảm kích thước mà còn giảm các features, giúp loại bỏ sự dư thừa. Quá trình PCA bao gồm việc tính toán ma trận phương sai, phân tích trị riêng, và chiếu lên không gian con có không gian nhỏ hơn.

Feature Selection Methods: Ngoài PCA, các phương pháp lựa chọn đặc trưng khác như mutual information-based selection, variance-based selection, hay correlation-based selection cũng có thể được áp dụng để lựa chọn tập con của features quan trọng nhất.



Hình 1.11: Biểu đồ so sánh hình dạng các hình chuẩn hóa khác nhau

1.2.1.3. Optimization và Acceleration

Batch Processing: Trong các ứng dụng CBIR thực tế, việc xử lý từng ảnh một có thể không hiệu quả về mặt tính toán. Batch processing cho phép xử lý nhiều ảnh cùng lúc, tận dụng tối đa khả năng xử lý song song của GPU. Kích thước batch cần được điều chỉnh dựa trên khả năng sử dụng bộ nhớ và tài nguyên tính toán. Batch size lớn hơn thường mang lại hiệu quả tính toán cao hơn nhưng đòi hỏi nhiều bộ nhớ hơn.

GPU Acceleration: là yếu tố then chốt trong việc triển khai các hệ thống CBIR large-scale. Các hoạt động trong CNN như convolution, phép nhân ma trận có thể xử lý song song hiệu quả trên GPU. Việc sử dụng frameworks như CUDA, cuDNN, và các deep learning libraries như PyTorch, TensorFlow giúp tối ưu hóa việc sử dụng GPU. Đối với sự suy luận, việc sử dụng mixed precision (FP16 thay vì FP32) có thể tăng tốc độ suy luận lên đáng kể mà không ảnh hưởng nhiều đến độ chính xác. TensorRT của NVIDIA là một công cụ mạnh mẽ để tối ưu hóa models học sâu cho suy luận.

Một vài models phổ biến cho việc tối ưu hóa:

- **Quantization:** Kỹ thuật quantization giúp giảm độ chính xác của trọng số và kích hoạt từ 32-bit floating point xuống 8-bit integers, từ đó giảm đáng kể model size và tăng tốc độ suy luận. Post-training quantization và quantization-aware training là hai phương pháp chính được sử dụng.
- **Pruning:** Model pruning loại bỏ các kết nối hoặc nơ ron ít quan trọng trong mạng neural, giúp giảm độ phức tạp tính toán mà vẫn duy trì

được độ chính xác. Structured pruning và unstructured pruning là hai loại pruning phổ biến.

- **Knowledge Distillation:** Kỹ thuật này train một model nhỏ hơn (student model) để bắt chước hành vi của một model lớn hơn (teacher model). Student model có thể đạt được performance gần với teacher model nhưng với chi phí tính toán thấp hơn đáng kể.

1.2.2. So sánh độ tương tự ảnh

1.2.2.1. Similarity metrics cho deep features

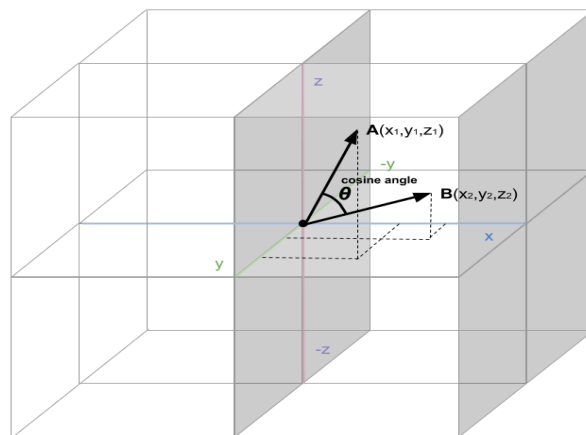
1.2.2.1.1. Cosine Similarity

Cosine Similarity là một trong những thước đo, chỉ số đo lường được sử dụng rộng rãi nhất trong CBIR với những đặc trưng nâng cao. Thước đo này đo lường góc giữa hai vectors trong không gian đa chiều, từ đó đánh giá độ tương tự về hướng mà không phụ thuộc vào kích thước của vectors.

Công thức của cosine similarity được định nghĩa như sau:

$$\cos(\theta) = (A \cdot B) / (\|A\| \times \|B\|)$$

Trong đó A và B là hai vectors đặc trưng, $A \cdot B$ là dot product, $\|A\|$ và $\|B\|$ là kích thước của các vectors tương ứng



Hình 1.12: Minh họa việc áp dụng cosine similarity trong không gian nhiều chiều

Ưu điểm của cosine similarity là khả năng bất biến với tỉ lệ, nghĩa là nó không bị ảnh hưởng bởi độ lớn của giá trị đặc trưng mà chỉ quan tâm đến kiểu mẫu và sự phân bố của các đặc trưng. Điều này đặc biệt quan trọng khi làm việc với những đặc trưng nâng cao, vì các giá trị của các đặc trưng có thể có các tỉ lệ khác nhau tùy thuộc vào kiến trúc hoạt động và quá trình training.

Khi các đặc trưng được L2-normalized (như thường được thực hiện trong training), cosine similarity trở nên tương đồng với dot product (tích vô hướng), giúp tăng tốc độ tính toán đáng kể. Điều này làm cho cosine similarity trở thành lựa chọn được ưu tiên hơn cho các ứng dụng real-time.

1.2.2.1.2. Euclidean Distance

Euclidean Distance là thước đo truyền thống đo lường khoảng cách thẳng trong không gian Euclidean giữa hai điểm. Khác với cosine similarity, Euclidean distance nhạy cảm với kích thước của các đặc trưng, điều này có thể vừa là ưu điểm vừa là nhược điểm. Trong một số trường hợp, kích cỡ của lượng thông tin có thể mang ý nghĩa quan trọng, ví dụ như cường độ của một vài đặc trưng nhất định. Tuy nhiên, trong các trường hợp khác, sự khác biệt về mặt tỉ lệ có thể gây ra nhiễu và làm giảm hiệu suất truy xuất ảnh.

Một biến thể của nó là Squared Euclidean Distance (khoảng cách Euclidean bình phương), loại bỏ phương thức căn bậc 2 trong công thức của Euclidean Distance để tăng tốc độ tính toán mà vẫn giữ nguyên thứ tự so sánh giữa các khoảng cách.

1.2.2.1.3. Mahalanobis Distance

Là một tổng quát hóa của khoảng cách Euclid, có xét đến ma trận hiệp phương sai giữa các chiều dữ liệu. Công thức:

$$d(A, B) = \sqrt{(A-B)^T \cdot S^{-1} \cdot (A-B)}$$

Trong đó S là ma trận hiệp phương sai. Mahalanobis distance có khả năng xử lý các chiều dữ liệu có mối tương quan và độ phân tán khác nhau, đồng thời tự động gán trọng số cho từng đặc trưng dựa trên tính chất thống kê. Tuy nhiên, việc tính toán và lưu trữ ma trận S tốn tài nguyên, đặc biệt với dữ liệu có hàng ngàn chiều.

1.2.2.1.4. Learned Distance Metrics

Thay vì sử dụng những phương thức có sẵn, hệ thống có thể học được cách đo khoảng cách tốt nhất dựa trên dữ liệu được huấn luyện. Các phương pháp học khoảng cách (metric learning) thường học dựa trên một ma trận biến đổi, hoặc một mạng neural để biến đổi vector sang không gian mới nơi mà giúp những phương thức đo lường cơ bản như khoảng cách Euclidean trở nên hiệu quả hơn.

1.2.2.2. Advanced Similarity Learning

Siamese Networks là một kiến trúc mạng được thiết kế đặc biệt để học các hàm so sánh độ tương tự. Kiến trúc bao gồm hai nhánh giống nhau, mỗi nhánh xử lý một ảnh trong cặp ảnh. Kết quả từ hai nhánh được kết hợp lại để tính độ tương tự. Kiến trúc này đặc biệt hiệu quả cho các bài toán ít dữ liệu, nơi hệ thống cần được học để nhận dạng độ tương đồng với những bài toán có dữ liệu nhỏ để huấn luyện.

Quá trình huấn luyện thường sử dụng contrastive loss hoặc binary cross – entropy để tối ưu hóa hệ thống trong việc phân biệt ảnh giống và khác nhau.

Trong CBIR, mạng Siamese có thể được huấn luyện trên các cặp ảnh giống và khác nhau để học hàm so sánh độ tương đồng sao cho phù hợp với từng lĩnh vực như ảnh y tế, nghệ thuật, thời trang – nơi mà các đặc trưng phổ biến không đủ để phân biệt.

Triplet loss là một hàm mất được thiết kế trong học nhúng ảnh, việc huấn luyện hàm này yêu cầu ba ảnh:

- Anchor (gốc).
- Positive (giống).
- Negative (khác).

Mục tiêu của hàm này để thu nhỏ khoảng cách giữa anchor và positive hơn negative. Triplet loss đặc biệt hiệu quả trong việc học nhúng phân biệt và được sử dụng rộng rãi trong việc nhận diện khuôn mặt, nhận dạng người và truy vấn ảnh. Tuy nhiên, việc lựa chọn bộ ba ảnh trong quá trình huấn luyện là một thách thức lớn và đòi hỏi sự phân tích tỉ mỉ.

Contrastive Learning (học tương phản) là một phương pháp học tự giám sát đang được chú ý. Ý tưởng của phương pháp này là học biểu diễn ảnh bằng cách đối chiếu các cặp ảnh giống nhau với các cặp ảnh khác nhau mà không cần nhãn. Những kỹ thuật mạnh mẽ như SimCLR, MoCo và SwAV đã chứng minh khả năng học đặc trưng mạnh mẽ dựa trên phương pháp này. Trong CBIR, contrastive

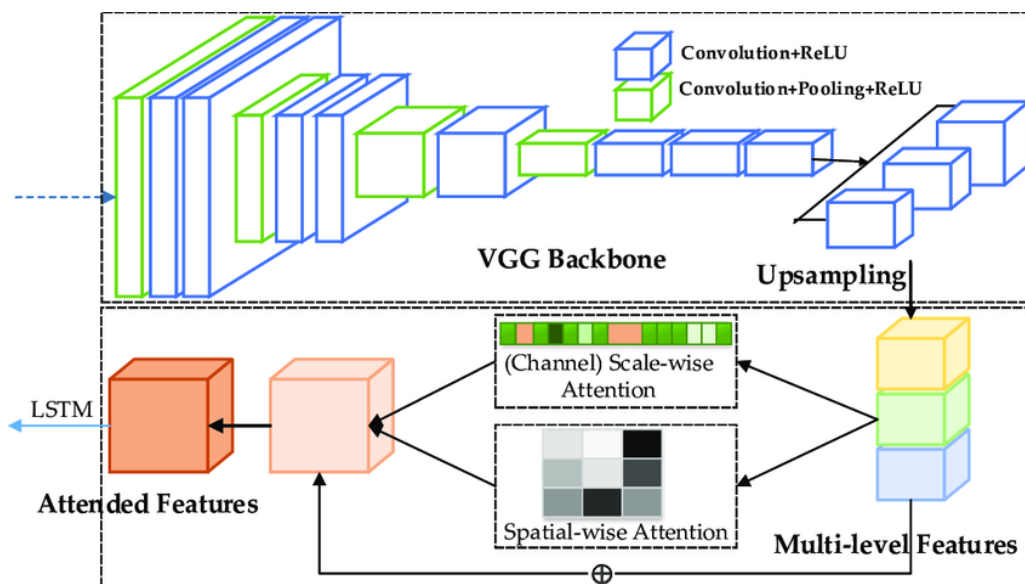
learning có thể được áp dụng để học cách biểu diễn đặc trưng tốt hơn, đặc biệt trong việc truy xuất ảnh.

Việc tối ưu hóa dữ liệu đóng vai trò quan trọng trong phương thức học tương phản, trong đó các phiên bản tăng cường khác nhau của cùng một hình ảnh được coi là các cặp giống nhau. Hãy thiết kế các chiến lược tăng cường phù hợp cho các miền cụ thể có yếu tố chính quyết định thành công trong cách tiếp cận phương pháp này.

1.2.2.3. So sánh đa tầng và đa mức

So sánh phân cấp là việc sử dụng các đặc trưng phân tầng của CNN để nắm bắt các chi tiết từ bậc thấp tới các cấp ngữ nghĩa bậc cao. Các lớp đầu nắm bắt các chi tiết như texture, edges trong khi các lớp sâu hơn mã hóa thông tin ngữ nghĩa trừu tượng hơn như objects, scene. Phương thức cho việc so sánh phân cấp bao gồm:

- Sự kết hợp đặc trưng phân tầng với các trọng số khác nhau.
- So sánh theo chiến lược từ thô đến tinh (coarse – to – fine) trong đó các tính năng bậc cao được sử dụng để lọc ban đầu và các tính năng cấp thấp để xếp hạng cuối cùng.
- So sánh theo độ phân giải sử dụng các tính năng ở các thang đo khác nhau.



Hình 1.13: So sánh đa tầng (layer fusion, coarse - to - fine)

So sánh sự tương đồng dựa trên cơ chế Attention, cơ chế này cho phép hệ thống tập trung vào các vùng quan trọng trong ảnh khi tính toán độ tương đồng. Hai phương thức chính là tập trung vào không gian (spatial attention) có thể xác định vùng nào của ảnh là quan trọng, và tập trung vào kênh đặc trưng (channel attention) cân nhắc các đặc trưng khác nhau dựa trên độ tương quan của chúng. Trong CBIR, attention giúp hệ thống so sánh thông minh hơn bằng cách “nhìn đúng chỗ”.

So sánh sự tương đồng theo từng phần (Part – based Similarity), phương pháp này phân tách hình ảnh thành các phần hoặc vùng có ý nghĩa để thực hiện so sánh ở cấp độ bộ phận. Sự tương đồng dựa trên bộ phận đặc biệt có giá trị đối với các nhiệm vụ nhận dạng chi tiết, trong đó các đối tượng cùng loại có thể có các bộ phận giống nhau nhưng có hình thức tổng thể khác nhau, hoặc ngược lại, các loại

khác nhau có thể có một số bộ phận chung, ví dụ trong các bài toán nhận dạng chi tiết như phân biệt các loài chim, mẫu giày, các dòng xe...

Trong thực tế, việc triển khai so sánh đa cấp độ và đa thang đo đòi hỏi phải cân nhắc cẩn thận về hiệu quả tính toán:

- **Lọc tiến bộ (progressive filtering):** Sử dụng các tính năng thô để lọc số lượng lớn dữ liệu, sau đó chỉ áp dụng so sánh chi tiết tốn kém cho các dữ liệu triển vọng
- **Lập chỉ mục phân cấp (Hierarchical Indexing):** Xây dựng các chỉ mục đa cấp để hỗ trợ tìm kiếm đa thang hiệu quả
- **Phương pháp gần đúng (Approximation):** Sử dụng các kỹ thuật gần đúng để tăng tốc các phép tính tương tự phức tạp trong khi vẫn duy trì độ chính xác chấp nhận được

1.3. Kết luận

Chương này đã cung cấp một cái nhìn tổng quan và có hệ thống về công nghệ tìm kiếm ảnh tương tự dựa trên nội dung (Content-Based Image Retrieval - CBIR), từ những khái niệm cơ bản đến các phương pháp tiên tiến hiện đại.

Thông qua việc phân tích kỹ thuật CBIR, chúng ta đã thấy rõ sự phát triển mạnh mẽ từ các phương pháp truyền thống sử dụng đặc trưng thủ công (hand-crafted features) sang các giải pháp dựa trên học sâu. Các công đoạn xử lý ảnh cơ bản bao gồm tiền xử lý, trích xuất đặc trưng, so sánh độ tương tự và đánh giá kết quả đã được trình bày một cách có hệ thống, tạo nền tảng vững chắc cho việc hiểu rõ toàn bộ quy trình tìm kiếm ảnh.

Đặc biệt, việc nghiên cứu sâu về mô hình Convolutional Neural Network (CNN) và kiến trúc ResNet đã cho thấy tiềm năng to lớn của học sâu trong việc giải quyết bài toán tìm kiếm ảnh tương tự. ResNet với cơ chế kết nối tắt (skip connections) không chỉ giải quyết hiệu quả vấn đề vanishing gradient mà còn cho phép xây dựng các mạng sâu hơn, từ đó nâng cao đáng kể khả năng trích xuất đặc trưng có ý nghĩa từ ảnh.

Tổng kết lại, chương này đã xây dựng được một framework lý thuyết toàn diện cho công nghệ tìm kiếm ảnh tương tự, kết hợp giữa các kiến thức nền tảng về xử lý ảnh và những tiến bộ mới nhất trong lĩnh vực học sâu. Những kiến thức này sẽ là nền tảng quan trọng cho các chương tiếp theo, nơi chúng ta sẽ đi sâu vào việc thiết kế, triển khai và đánh giá hệ thống tìm kiếm ảnh tương tự trong thực tế.

CHƯƠNG 2. THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG TÌM KIẾM ẢNH THÔNG MINH

2.1. Tích hợp hệ thống CBIR vào ứng dụng web với mô hình ResNet – 50 và FAISS

Sau khi đã nghiên cứu chi tiết các phương pháp trích xuất đặc trưng và so sánh độ tương tự, bước tiếp theo là tích hợp các kỹ thuật này vào một hệ thống web hoàn chỉnh. Điều này đòi hỏi sự kết hợp hài hòa giữa mô hình học sâu, hệ thống lập chỉ mục hiệu quả và công nghệ web. Phần này sẽ trình bày quá trình triển khai một hệ thống CBIR hoàn chỉnh sử dụng ResNet-50 làm bộ trích xuất đặc trưng và Faiss – công cụ tìm kiếm độ tương tự do Facebook phát triển – làm engine tìm kiếm ảnh tương tự.

2.1.1. Kiến trúc tổng thể của hệ thống

Hệ thống tìm kiếm ảnh dựa trên nội dung (Content-Based Image Retrieval - CBIR) được thiết kế theo mô hình kiến trúc phân lớp (Layered Architecture) nhằm đảm bảo tính mô-đun, khả năng mở rộng và dễ bảo trì. Mỗi lớp có trách nhiệm riêng biệt và giao tiếp với nhau thông qua các giao diện được định nghĩa rõ ràng. Hệ thống được tổ chức theo 4 lớp chính, từ trên xuống dưới:

2.1.1.1. Lớp giao diện người dùng (Presentation Layer)

Cung cấp giao diện tương tác giữa người dùng và hệ thống

Thành phần chính:

- **Giao diện tải lên ảnh:** Cho phép người dùng upload ảnh truy vấn với các định dạng được hỗ trợ (JPEG, PNG, BMP)
- **Giao diện tìm kiếm:** Hiển thị các tùy chọn tìm kiếm và bộ lọc kết quả
- **Giao diện hiển thị kết quả:** Trình bày kết quả tìm kiếm theo dạng lưới với thông tin độ tương tự và metadata
- **Giao diện quản lý người dùng:** Xử lý đăng nhập, đăng ký và quản lý hồ sơ cá nhân

2.1.1.2. Lớp xử lý logic (Logic Layer)

Xử lý logic và điều phối luồng dữ liệu trong hệ thống

Thành phần chính:

- **Django Web Framework:** Khung phát triển web chính, quản lý routing và middleware
- **API Controllers:** Xử lý các HTTP requests và responses cho các chức năng CBIR
- **Service Layer:** Chứa logic nghiệp vụ cho tìm kiếm, quản lý ảnh và xử lý người dùng
- **Authentication & Authorization:** Quản lý phiên làm việc, xác thực và phân quyền người dùng

- **File Management Service:** Quản lý việc upload, lưu trữ và truy xuất file ảnh

2.1.1.3. Lớp mô hình học sâu (Deep Learning Processing Layer)

Thực hiện các tác vụ xử lý ảnh và trích xuất đặc trưng bằng deep learning

Thành phần chính:

- **Feature Extraction Module:**
 - Mô hình ResNet-50 pre-trained: Trích xuất đặc trưng deep learning từ ảnh input
 - Image Preprocessing: Chuẩn hóa kích thước, normalization và augmentation
 - Vector Extraction: Chuyển đổi ảnh thành vector đặc trưng có độ dài cố định
- **Similarity Search Engine:**
 - FAISS (Facebook AI Similarity Search): Thực hiện tìm kiếm k-nearest neighbors hiệu quả
 - Distance Metrics: Hỗ trợ các độ đo khoảng cách như Euclidean, Cosine similarity
 - Index Management: Quản lý và cập nhật chỉ mục vector

2.1.1.4. Lớp dữ liệu (Data Persistence Layer)

Quản lý lưu trữ và truy xuất dữ liệu của hệ thống

Thành phần chính:

- **Relational Database (SQLite):**
 - Lưu trữ metadata của ảnh (tên file, kích thước, thời gian upload, tags)
 - Quản lý thông tin người dùng và session
 - Lưu trữ lịch sử tìm kiếm và thống kê hệ thống
- **File Storage System:**
 - Lưu trữ ảnh gốc trong hệ thống file với cấu trúc thư mục được tổ chức
 - Thumbnail generation và caching
 - Backup và recovery strategy
- **Vector Index Storage:**
 - FAISS index files: Lưu trữ chỉ mục vector đặc trưng được tối ưu hóa
 - Index versioning và incremental updates
 - Distributed storage support cho scalability

2.1.2. Lựa chọn và tối ưu hóa ResNet - 50

ResNet-50 được chọn làm kiến trúc trích xuất đặc trưng cho hệ thống CBIR nhờ vào khả năng cân bằng tốt giữa độ chính xác và hiệu suất tính toán. Với 50 lớp, mô hình này đủ sâu để học được các đặc trưng thị giác phức tạp nhưng không quá nặng để triển khai trong môi trường thực tế. ResNet – 50 đã được đào tạo trước trên ImageNet với hơn 14 triệu ảnh, tạo ra một cơ sở biểu diễn trực quan phong phú. Kiến trúc sử dụng kết nối dư (residual connections) giúp tránh hiện tượng mất gradient, từ đó học được đặc trưng ở nhiều mức độ từ chi tiết thấp đến khái niệm ngữ nghĩa cao – rất phù hợp với các ứng dụng CBIR.

Trong hệ thống này, đặc trưng được trích xuất từ lớp pooling trung bình toàn cục ngay trước lớp phân loại cuối cùng, tạo ra vector đặc trưng có 2048 chiều. Việc chọn lớp này giúp giữ được tính ngữ nghĩa mạnh mà vẫn đảm bảo hiệu quả lưu trữ và truy xuất. Trước khi đưa ảnh vào mạng ResNet-50, ảnh được resize về 224x224 pixel, chuẩn hóa theo thống kê của ImageNet và có thể áp dụng tăng cường dữ liệu để tăng độ bền vững. Mô hình sau đó được chuyển sang định dạng tối ưu như TorchScript hoặc ONNX nhằm tăng tốc suy luận và giảm chi phí bộ nhớ.

Để đảm bảo trải nghiệm người dùng tốt trên môi trường web, mô hình được tối ưu thông qua các kỹ thuật như:

- **Rút gọn độ chính xác (quantization)** từ FP32 xuống FP16 hoặc INT8 giúp giảm kích thước và tăng tốc suy luận.
- **Xử lý hàng loạt (batch processing)** cho phép xử lý nhiều ảnh cùng lúc.
- **Cơ chế cache ở đa tầng:** Từ trọng số mô hình cho đến các đặc trưng truy cập thường xuyên, giúp giảm độ trễ.

2.1.3. Tích hợp Faiss cho truy vấn ảnh quy mô lớn

Faiss (Facebook AI Similarity Search) là thư viện tối ưu cho tìm kiếm độ tương tự và phân cụm vector đặc trưng dạng đặc (dense). Thư viện này hỗ trợ nhiều thuật toán lập chỉ mục từ truy vấn chính xác (exact search) đến truy vấn gần đúng (approximate search), cho phép điều chỉnh giữa tốc độ và độ chính xác. Faiss hỗ trợ cả CPU và GPU, phù hợp với hệ thống có quy mô từ vài nghìn đến hàng tỷ vector.

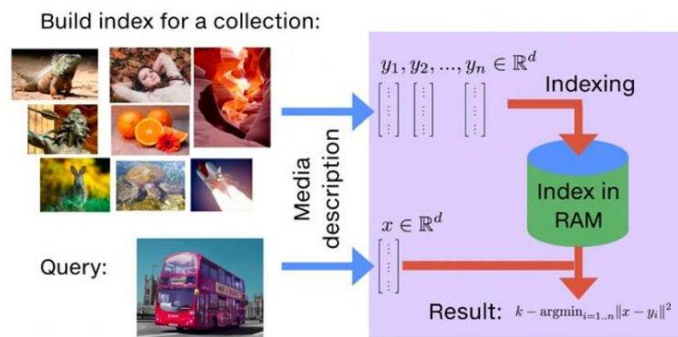
Các ưu điểm chính của Faiss bao gồm: cài đặt tối ưu cho các thuật toán tìm kiếm, hỗ trợ nhiều loại khoảng cách, sử dụng bộ nhớ hiệu quả, và khả năng mở rộng tốt – phù hợp với hệ thống CBIR cần xử lý nhanh và chính xác.

Kiến trúc hệ thống gồm nhiều thành phần hoạt động phối hợp:

- Dịch vụ trích xuất đặc trưng xử lý ảnh qua ResNet-50, tạo ra vector đặc trưng.
- Các vector đặc trưng được lưu trữ trong chỉ mục Faiss cùng với metadata như đường dẫn ảnh, nhãn phân loại, ...
- Dịch vụ tìm kiếm tiếp nhận ảnh truy vấn, thực hiện tìm kiếm độ tương tự và trả về danh sách kết quả đã sắp xếp.

Thông tin metadata được lưu trong hệ quản trị cơ sở dữ liệu, trong khi ảnh gốc có thể được lưu trên hệ thống tệp hoặc đám mây. API web cung cấp các điểm truy cập RESTful để phục vụ thao tác tải ảnh, tìm kiếm, và hiển thị kết quả. Hệ thống được thiết kế với các cơ chế cân bằng tải và cache để đảm bảo hiệu năng khi có nhiều người dùng đồng thời.

Đối với hệ thống quy mô lớn, kiến trúc phân tán là điều cần thiết. Faiss hỗ trợ chia nhỏ chỉ mục và phân phối lên nhiều máy chủ (index sharding), giúp mở rộng theo chiều ngang. Việc sử dụng GPU mang lại lợi thế lớn trong cả trích xuất đặc trưng lẫn tìm kiếm, cho phép xử lý song song theo lô (batch processing). Bộ nhớ cũng được tối ưu kỹ lưỡng để phù hợp với các chỉ mục lớn.



Hình 2.14: Tích hợp service extraction, index Faiss và metadata

2.2. Kiến trúc và triển khai ứng dụng Web

2.2.1. Thành phần giao diện người dùng (Frontend)

Phần frontend được xây dựng bằng các framework hiện đại như JavaScript để cung cấp giao diện thân thiện cho người dùng. Chức năng kéo và thả giúp người dùng dễ dàng tải ảnh truy vấn, còn cơ chế tải kết quả dần (progressive loading) mang lại trải nghiệm mượt mà.

Thư viện hiển thị ảnh truy vấn và kết quả với thông tin như điểm tương đồng, nhãn phân loại và dữ liệu liên quan. Người dùng có thể lọc kết quả theo danh mục, ngày tải lên hoặc các tiêu chí khác. Hệ thống cũng cung cấp gợi ý tìm kiếm thời gian thực.

2.2.1.1. Xây dựng Home Page

```
<body>
  <div class="background"></div>

  <header>
    <div class="logo"></div>

    {%if user.is_authenticated %}
    <div class="user-menu" id="userMenu">
      <div class="user-avatar">{{user.username.0|upper}}</div>
      <span style="color: white; font-weight: bold; font-size: 1.2em;">{{user.username}}</span>
      <div class="dropdown-menu" id="dropdownMenu">
        <a href="#" class="dropdown-item">Profile</a>
        <a href="{% url 'search_history' %}" class="dropdown-item">My Searches</a>
        <a href="#" class="dropdown-item">Settings</a>
        <div class="dropdown-divider"></div>
        <a href="{% url 'logout' %}" class="dropdown-item">Logout</a>
      </div>
    </div>
    </div>

    {% else %}
    <div class="nav-buttons">
      <button class="btn btn-white" id="registerbtn">Register</button>
      <button class="btn btn-yellow" id="loginbtn">
        <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16" viewBox="0 0 24 24" fill="none"
          stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round">
            <path d="M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4-4v2"></path>
            <circle cx="12" cy="7" r="4"></circle>
          </svg>
        Log in
      </button>
    </div>
    {% endif %}
  </header>

  <div class="location-text">Hanoi</div>
```

```
<main>
  {% if message %}
  {% for message in messages %}
  <div class="welcome-message" id="welcomeMessage">
    {{message}}
  </div>
  {% endfor %}
  {% endif %}

  <div class="search-container">
    <input type="text" class="search-input" placeholder="Search...">
    <button class="camera-button" id="cameraBtn">
      <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16" viewBox="0 0 24 24" fill="none"
        stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round">
          <path d="M23 19a2 2 0 0 1-2 2H3a2 2 0 0 1-2-2V8a2 2 0 0 1 2-2h4l2-3h6l2 3h4a2 2 0 0 1 2 2z"></path>
          <circle cx="12" cy="13" r="4"></circle>
        </svg>
      </button>
    <button class="search-button">
      <svg xmlns="http://www.w3.org/2000/svg" width="20" height="20" viewBox="0 0 24 24" fill="none"
        stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round">
          <circle cx="11" cy="11" r="8"></circle>
          <line x1="21" y1="21" x2="16.65" y2="16.65"></line>
        </svg>
      </button>
    </div>
  </main>

  <div class="footer-space"></div>
```

```

<div class="modal" id="uploadModal">
  <div class="modal-content">
    <h3>Visual search</h3>
    <div class="upload-area" id="dropArea">
      
      <p class="text-image"> Drag an image here</p>
      <p class="upload-text">or click the button</p>
      <div class="button-group">
        <button class="upload-btn" id="uploadBtn">Select file</button>
        <button class="paste-btn" id="pasteBtn">Paste from clipboard</button>
      </div>
    </div>
    <form id="imageSearchForm" method="post" action="{% url 'search_by_image' %}" enctype="multipart/form-data"
      style="display: none;">
      {% csrf_token %}
      <input type="file" id="imageFile" name="image" accept="image/*">
      <button type="submit" id="submitImageSearch">Search</button>
    </form>
  </div>
</div>

```

Xây dựng khung HTML cho HomePage với những thành phần chính:

- Thanh Search kết hợp nút bấm tìm kiếm ảnh.
- Nút bấm chuyển hướng tới trang đăng ký và đăng nhập.
- Sau khi đăng nhập có phần profile để truy xuất lịch sử tìm kiếm.


```

<script>

// Register Btn (chỉ chạy khi element tồn tại)
const registerBtn = document.getElementById("registerbtn");
if (registerBtn) {
  registerBtn.addEventListener("click", function () {
    window.location.href = "{% url 'register' %}";
  });
}

// Login Btn (chỉ chạy khi element tồn tại)
const loginBtn = document.getElementById("loginbtn");
if (loginBtn) {
  loginBtn.addEventListener("click", function () {
    window.location.href = "{% url 'login' %}";
  });
}

// JavaScript to handle modal functionality
const cameraBtn = document.getElementById('cameraBtn');
const modal = document.getElementById('uploadModal');
const dropArea = document.getElementById('dropArea');
const uploadBtn = document.getElementById('uploadBtn');
const pasteBtn = document.getElementById('pasteBtn');

// Open modal when camera button is clicked
if (cameraBtn && modal) {
  cameraBtn.addEventListener('click', function (e) {
    console.log('Camera button clicked!');
    e.stopPropagation();
    modal.classList.add('active');
  });
}

```

```

// Handle upload button click - only trigger file selection
if (uploadBtn) {
  uploadBtn.addEventListener('click', function (e) {
    e.stopPropagation(); // Prevent event bubbling
    document.getElementById('imageFile').click();
  });
}

// Handle drop area click - only for drag and drop, not button clicks
if (dropArea) {
  dropArea.addEventListener('click', function (e) {
    // Check if the click target is NOT one of the buttons
    if (!e.target.classList.contains('upload-btn') &&
        !e.target.classList.contains('paste-btn')) {
      document.getElementById('imageFile').click();
    }
  });
}

// Handle file selection
const imageFile = document.getElementById('imageFile');
if (imageFile) {
  imageFile.addEventListener('change', function () {
    if (this.files.length > 0) {
      const submitBtn = document.getElementById('submitImageSearch');
      if (submitBtn) {
        submitBtn.click();
      }
      if (modal) {
        modal.classList.remove('active');
      }
    }
  });
}

```

Xây dựng JavaScript xử lý logic cho các nút và thanh công cụ.

2.2.1.2. Xây dựng trang Login

```
<body>
  <div class="login-container">
    <h1>Login</h1>

    {% if messages %}
    <div id="messageContainer">
      {% for message in messages %}
        <div class="general-message {% if message.tags %}{{ message.tags }}{% endif %}">
          {{ message }}
        </div>
      {% endfor %}
    </div>
    {% else %}
    <div id="messageContainer"></div>
    {% endif %}

    <form method="post" action="{% url 'login' %}">
      {% csrf_token %}

      <div class="form-group">
        <label for="username">Username or Email</label>
        <input type="text" id="username" name="username" placeholder="Enter your username or email">
        {% if form.username.errors %}
        <p class="error-message" id="usernameError">{{ form.username.errors.0 }}</p>
        {% endif %}
      </div>

      <div class="form-group">
        <label for="password">Password</label>
        <input type="password" id="password" name="password" placeholder="Enter your password">
        {% if form.password.errors %}
        <p class="error-message" id="passwordError">{{ form.password.errors.0 }}</p>
        {% endif %}
      </div>

      <button type="submit" class="button">Login</button>
    </form>

    <div class="additional-links">
      <a href="{% url 'password_reset' %}">Forgot Password?</a>
      <a href="{% url 'register' %}">Register</a>
    </div>
  </div>
```

Xây dựng trang login bao gồm hai trường chính là username và password đã đăng kí, hiển thị tin nhắn báo lỗi cũng như thành công.

```

<script>
document.addEventListener('DOMContentLoaded', function() {
  const form = document.querySelector('form');
  const loginButton = document.querySelector('.button');

  loginButton.addEventListener('click', function(e) {
    e.preventDefault();
    const username = document.getElementById('username').value;
    const password = document.getElementById('password').value;

    clearErrors();

    let hasErrors = false;

    if(!username) {
      showError('username', 'Username or email is required');
      hasErrors = true;
    }

    if(!password) {
      showError('password', 'Password is required');
      hasErrors = true;
    }

    if (!hasErrors) {
      form.submit();
    }
  });

  function showError(field, message) {
    const inputElement = document.getElementById(field);

    // Check if error element already exists
    let errorElement = document.getElementById(field + 'Error');

    if (!errorElement) {
      errorElement = document.createElement('p');
      errorElement.id = field + 'Error';
      errorElement.className = 'error-message';
      inputElement.parentNode.appendChild(errorElement);
    }

    errorElement.textContent = message;
    inputElement.style.borderColor = '#ff3333';
  }

  function showGeneralMessage(message, type) {

```

Xây dựng JS xử lý cho việc submit cũng như thể hiện thông báo lỗi.

2.2.1.3. Xây dựng RegisterPage

```
(body)
<div class="container">
  <div class="title">Registration</div>
  {% if messages %}
    {% for message in messages %}
      <div class="alert" {% if message.tags %}alert-{{ message.tags }}{% endif %}>
        {{ message }}
      </div>
    {% endfor %}
  {% endif %}
  <div id="successMessage" {% if form_submitted %}style="display:block;"{% endif %}>Registration successful! Redirecting to login page...</div>
  <div class="content">
    <!-- Registration form -->
    <form method="post" id="registrationForm">
      {% csrf_token %}
      <div class="user-details">
        <!-- Input for Full Name -->
        <div class="input-box">
          <span class="details">Full Name</span>
          <input type="text" id="fullName" name="full_name" placeholder="Enter your name" value="{{ form.full_name.value|default:'' }}" required>
          <span class="error" id="fullNameError">{{ form.full_name.errors }}{{ form.username.errors }}{{ form.email.errors }}{{ form.phone.errors }}{{ form.password1.errors }}{{ form.password2.errors }}{{ form.gender.errors }}
```

Triển khai những thuộc tính cơ bản như fullname, username, password, email, số điện thoại, trường chọn giới tính.

```

<script>
document.addEventListener('DOMContentLoaded', function() {
    function validateForm() {

        // Reset errors
        const errorElements = document.querySelectorAll('.error');
        errorElements.forEach(element => {
            element.textContent = '';
        });

        // Validate Full Name
        const fullName = document.getElementById('fullName').value.trim();
        if (fullName === '') {
            document.getElementById('fullNameError').textContent = 'Full name is required';
            isValid = false;
        }

        // Validate Username
        const username = document.getElementById('username').value.trim();
        if (username === '') {
            document.getElementById('usernameError').textContent = 'Username is required';
            isValid = false;
        } else if (username.length < 4) {
            document.getElementById('usernameError').textContent = 'Username must be at least 4 characters';
            isValid = false;
        }

        // Validate Email
        const email = document.getElementById('email').value.trim();
        const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
        if (email === '') {
            document.getElementById('emailError').textContent = 'Email is required';
            isValid = false;
        } else if (!emailRegex.test(email)) {
            document.getElementById('emailError').textContent = 'Please enter a valid email';
            isValid = false;
        }
    }
}

```

```

        // Validate Phone
        const phone = document.getElementById('phone').value.trim();
        const phoneRegex = /^d{10,15}$/;
        if (phone === '') {
            document.getElementById('phoneError').textContent = 'Phone number is required';
            isValid = false;
        } else if (!phoneRegex.test(phone.replace(/[-() \s]/g, ''))) {
            document.getElementById('phoneError').textContent = 'Please enter a valid phone number';
            isValid = false;
        }

        // Validate Password
        const password = document.getElementById('password').value;
        if (password === '') {
            document.getElementById('passwordError').textContent = 'Password is required';
            isValid = false;
        } else if (password.length < 8) {
            document.getElementById('passwordError').textContent = 'Password must be at least 8 characters';
            isValid = false;
        }

        // Validate Confirm Password
        const confirmPassword = document.getElementById('confirmPassword').value;
        if (confirmPassword === '') {
            document.getElementById('confirmPasswordError').textContent = 'Please confirm your password';
            isValid = false;
        } else if (confirmPassword !== password) {
            document.getElementById('confirmPasswordError').textContent = 'Passwords do not match';
            isValid = false;
        }

        // Validate Gender
        const gender = document.querySelector('input[name="gender"]:checked');
        if (!gender) {
            document.getElementById('genderError').textContent = 'Please select a gender';
            isValid = false;
        }

        return isValid;
    }
}

```

JavaScript xử lý kiểm tra thông tin của từng trường xem đã hợp lý chưa và báo lỗi nếu chưa, thành công thì lưu vào cơ sở dữ liệu.

2.2.1.4. Xây dựng ResultPage

```
<body>
  <!-- Header -->
  <header class="header">
    <div class="header-content">
      <a href="/" class="logo">
        <i class="fas fa-search"></i>
      </a>
      <div class="search-info">
        {% if total_results %}
          <span class="results-count">{{ total_results }} kết quả</span>
        {% endif %}
      </div>
    </div>
  </header>

  <div class="container">
    <!-- Query Image Section -->
    {% if original_image %}
      <section class="query-section">
        <div class="query-header">
          <i class="fas fa-image"></i>
          <h2>Ảnh tìm kiếm</h2>
        </div>
        <div class="query-image-container">
          
          <div class="query-details">
            <div class="detail-item">
              <span class="detail-label">Tên file:</span>
              <span class="detail-value">{{ query_image_name|default:"unknown.jpg" }}</span>
            </div>
            {% if image_info %}
              {% if image_info.width and image_info.height %}
                <div class="detail-item">
                  <span class="detail-label">Kích thước:</span>
                  <span class="detail-value">{{ image_info.width }} x {{ image_info.height }}px</span>
                </div>
              {% endif %}
              {% if image_info.file_size_mb %}
                <div class="detail-item">
                  <span class="detail-label">Dung lượng:</span>
                  <span class="detail-value">{{ image_info.file_size_mb }} MB</span>
                </div>
              {% endif %}
              {% if image_info.format %}
                <div class="detail-item">
                  <span class="detail-label">Định dạng:</span>
                  <span class="detail-value">{{ image_info.format|upper }}</span>
                </div>
              {% endif %}
            {% endif %}
          </div>
        </div>
      </section>
    {% endif %}
  </div>
```

```

<!-- Filters -->
<section class="filters">
  <div class="filter-group">
    <div class="filter-item">
      <label for="similarity">Độ tương đồng:</label>
      <select id="similarity" class="filter-select">
        <option value="all">Tất cả</option>
        <option value="high">Cao (>80%)</option>
        <option value="medium">Trung bình (50-80%)</option>
        <option value="low">Thấp (<50%)</option>
      </select>
    </div>
    <div class="filter-item">
      <label for="sort">Sắp xếp:</label>
      <select id="sort" class="filter-select">
        <option value="similarity">Độ tương đồng</option>
        <option value="name">Tên file</option>
        <option value="recent">Mới nhất</option>
      </select>
    </div>
    <div class="filter-item">
      <label for="density">Mật độ hiển thị:</label>
      <select id="density" class="filter-select">
        <option value="normal">Bình thường</option>
        <option value="compact">Nén</option>
        <option value="ultra-compact">Siêu nén</option>
      </select>
    </div>
  </div>
</section>

<!-- Results Section -->
<section class="results-section">
  <div class="results-header">
    <h2 class="results-title">
      Ảnh tương tự
      {% if total_results %}({{ total_results }} kết quả){% endif %}
    </h2>
    <div class="view-toggle">
      <button class="view-btn active" data-view="grid">
        <i class="fas fa-th"></i> Lưới
      </button>
      <button class="view-btn" data-view="list">
        <i class="fas fa-list"></i> Danh sách
      </button>
    </div>
  </div>
</div>

```

```

{% if search_results %}
<div class="image-grid" id="imageGrid">
  {% for result in search_results %}
    <div class="image-card" data-similarity="{{ result.similarity_percent }}">
      <div class="image-container">
        
        <div class="similarity-badge">{{ result.similarity_percent }}%</div>
        <div class="image-overlay">
          <h3>{{ result.title }}</h3>
          <p>Độ tương đồng: {{ result.similarity_percent }}%</p>
        </div>
      </div>
      <div class="image-info">
        <h3 class="image-title">{{ result.title }}</h3>
        <div class="image-meta">
          <span class="source-tag">{{ result.source|default:"Dataset" }}</span>
          <span>{{ result.similarity_percent }}% tương đồng</span>
        </div>
        <div class="action-buttons">
          <button class="action-btn btn-primary" onclick="viewImage('{{ result.image_url }}')">
            <i class="fas fa-eye"></i> Xem
          </button>
        </div>
      </div>
    </div>
  {% endfor %}
</div>

{% else %}
<div class="no-results">
  <i class="fas fa-image"></i>
  <h3>Không tìm thấy kết quả</h3>
  <p>Hãy thử với ảnh khác hoặc điều chỉnh ngưỡng tương đồng</p>
</div>
{% endif %}
</section>
</div>

```

Triển khai trang kết quả với từng card ảnh giúp hiển thị ảnh tương đồng cũng như, mục header hiển thị thông số kỹ thuật của ảnh upload.

Xây dựng JavaScript nhằm xử lý logic cho từng card ảnh hiển thị từ việc tìm ảnh tương tự.

2.2.1.5. Xây dựng SearchHistoryPage

```
<div class="container">
  <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav">
    <span class="navbar-toggler-icon"></span>
  </button>
  <div class="collapse navbar-collapse" id="navbarNav">
    <ul class="navbar-nav ms-auto">
      <li class="nav-item">
        <a class="nav-link" href="{% url 'home' %}">Trang chủ</a>
      </li>
      <li class="nav-item">
        <a class="nav-link active" href="{% url 'search_history' %}">Lịch sử tìm kiếm</a>
      </li>
    </ul>
  </div>
</div>
</nav>

<div class="container my-5">
  <!-- Statistics Section -->
  <div class="search-stats">
    <div class="row">
      <div class="col-md-4">
        <div class="stat-item">
          <span class="stat-number">{{ total_searches }}</span>
          <div class="stat-label">Tổng số tìm kiếm</div>
        </div>
      </div>
      <div class="col-md-4">
        <div class="stat-item">
          <span class="stat-number">{{ image_searches_count }}</span>
          <div class="stat-label">Tìm kiếm bằng hình ảnh</div>
        </div>
      </div>
      <div class="col-md-4">
        <div class="stat-item">
          <span class="stat-number">{{ text_searches_count }}</span>
          <div class="stat-label">Tìm kiếm bằng văn bản</div>
        </div>
      </div>
    </div>
  </div>
</div>
```

```
<div class="card-body">
  <h6 class="card-title mb-2">
    {% if history.is_image_search %}
      Tìm kiếm hình ảnh
    {% else %}
      {{ history.query|truncatechars:30 }}
    {% endif %}
  </h6>

  <div class="d-flex justify-content-between align-items-center mb-2">
    <small class="search-date">
      <i class="fas fa-clock me-1"></i>
      {{ history.created_at|date:"d/m/Y H:i" }}
    </small>
  </div>

  <div class="d-flex gap-2">
    {% if history.is_image_search and history.image_path %}
      <!-- Nút tìm kiếm lại với cùng hình ảnh -->
      <button class="btn btn-outline-primary btn-sm flex-fill"
        data-image-path="{{ history.image_path }}" onclick="searchAgain(this.dataset.imagePath)">
        <i class="fas fa-redo me-1"></i>Tìm lại
      </button>
    {% endif %}

    <!-- Nút xóa lịch sử -->
    <button class="btn btn-outline-danger btn-sm data-history-id="{{ history.id }}"
      onclick="deleteHistory(this.dataset.historyId)">
      <i class="fas fa-trash"></i>
    </button>
  </div>
</div>
</div>
{% endfor %}
</div>
```

```

<!-- Filter Tabs -->
<div class="filter-tabs">
  <a href="?type=all"
    class="filter-tab {% if not request.GET.type or request.GET.type == 'all' %}active{% endif %}">
    <i class="fas fa-list me-1"></i>Tất cả
  </a>
  <a href="?type=image" class="filter-tab {% if request.GET.type == 'image' %}active{% endif %}">
    <i class="fas fa-image me-1"></i>Hình ảnh
  </a>
  <a href="?type=text" class="filter-tab {% if request.GET.type == 'text' %}active{% endif %}">
    <i class="fas fa-font me-1"></i>Văn bản
  </a>
</div>

<!-- Search History Grid -->
{% if search_histories %}
<div class="row">
  {% for history in search_histories %}
  <div class="col-lg-3 col-md-4 col-sm-6 mb-4">
    <div class="card search-history-card h-100">
      <div class="position-relative">
        {% if history.is_image_search and history.image_url and history.image_exists %}
        
        {% elif history.is_image_search %}
        <div class="search-image-placeholder">
          <i class="fas fa-image"></i>
        </div>
        {% else %}
        <div class="search-image-placeholder">
          <i class="fas fa-font"></i>
        </div>
        {% endif %}

        <span
          class="badge {% if history.is_image_search %}bg-primary{% else %}bg-success{% endif %} search-type-badge">
          {% if history.is_image_search %}
          <i class="fas fa-image me-1"></i>Hình ảnh
          {% else %}
          <i class="fas fa-font me-1"></i>Văn bản
          {% endif %}
        </span>
      </div>

      <div class="card-body">
        <h6 class="card-title mb-2">
          {% if history.is_image_search %}
          Tìm kiếm hình ảnh
          {% else %}
          {{ history.query|truncatechars:30 }}
          {% endif %}
        </h6>

        <div class="d-flex justify-content-between align-items-center mb-2">
          <small class="search-date">
            <i class="fas fa-clock me-1"></i>

```

Thiết kế trang hiển thị lịch sử tìm kiếm người dùng với ảnh được lấy từ thư mục /media/searchimage được lưu lại sau mỗi lần upload.

```

function deleteHistory(historyId) {
  if (confirm('Bạn có chắc chắn muốn xóa lịch sử tìm kiếm này?')) {
    fetch(`/api/delete-search-history/${historyId}/`, {
      method: 'DELETE',
      headers: {
        'X-CSRFToken': '{{ csrf_token }}',
        'Content-Type': 'application/json',
      },
    })
      .then(response => response.json())
      .then(data => {
        if (data.status === 'success') {
          location.reload();
        } else {
          alert('Có lỗi xảy ra khi xóa lịch sử tìm kiếm.');

```

JavaScript xử lý hiển thị ảnh trong database lên những card ảnh đồng thời xử lý hàm xóa lịch sử người dùng.

2.2.2. Thiết kế API Backend với Django

Django được chọn làm backend nhờ hệ sinh thái mạnh mẽ và phù hợp với các ứng dụng CBIR. Django REST Framework (DRF) giúp xây dựng các API RESTful nhanh chóng, bảo mật và có khả năng mở rộng.

Kiến trúc API được thiết kế dựa trên mô hình Model-View-Serializer của Django:

- Model quản lý dữ liệu ảnh, đặc trưng và lịch sử truy vấn.
- Serializer chịu trách nhiệm kiểm tra và chuyển đổi dữ liệu.
- ViewSet xử lý logic và phản hồi HTTP.

Hệ thống có các endpoint chính như:

- Gửi ảnh lên.
- Truy vấn ảnh tương tự.
- Lấy kết quả truy vấn đã tính trước.
- Truy xuất lịch sử tìm kiếm người dùng.

Django hỗ trợ xác thực người dùng thông qua JWT để đảm bảo tính bảo mật. Các lớp quyền riêng biệt đảm bảo chỉ người dùng hợp lệ mới truy cập được ảnh và lịch sử của mình. Đồng thời, middleware chống lạm dụng API được cài đặt để đảm bảo tài nguyên hệ thống được phân phối công bằng.

Mô hình ResNet-50 được tải và sử dụng trong quá trình khởi động hệ thống Django để đảm bảo không phải tải lại nhiều lần. Các tác vụ trích xuất đặc trưng được xử lý ở chế độ không đồng bộ bằng Celery, giúp giảm tải thời gian phản hồi trên giao diện người dùng.

Cơ sở dữ liệu được thiết kế với ORM của Django, dễ bảo trì và có hiệu năng tốt. Các bảng chính bao gồm:

- Hình ảnh tải lên.
- Vector đặc trưng tương ứng.
- Lịch sử tìm kiếm của người dùng.

Django migrations được dùng để kiểm soát phiên bản cơ sở dữ liệu, giúp triển khai dễ dàng và nhất quán.

2.2.2.1. Xây dựng module quản lý tài khoản người dùng (Accounts)

2.2.2.1.1. Thiết kế Model người dùng

```
SearchImage > accounts > models.py > ...
1  from django.db import models
2  from django.contrib.auth.models import User
3
4  class UserProfile(models.Model):
5      user = models.OneToOneField(User, on_delete=models.CASCADE, related_name='profile')
6
7      phone_number = models.CharField(max_length=15)
8
9      GENDER_CHOICES = [
10         ('M', 'Male'),
11         ('F', 'Female'),
12         ('N', 'Prefer not to say'),
13     ]
14
15     gender = models.CharField(max_length=1, choices=GENDER_CHOICES)
16
17     def __str__(self):
18         return f"{self.user.username}'s Profile"
19
```

Model UserProfile mở rộng thông tin cho người dùng Django mặc định (User).

Trước tiên import module “models” để dùng các lớp như CharField, OneToOneField. Đồng thời import model “user” mặc định của Django chứa các thuộc tính mặc định như username, email, password.

Dòng 6: Thể hiện liên kết 1 – 1 với model User (mỗi người dùng chỉ có 1 profile), “on_delete = models.CASCADE” – nếu User bị xóa thì UserProfile cũng bị xóa theo.

Dòng 7: Trường ký tự lưu số điện thoại, giới hạn tối đa 15 ký tự.

Dòng 9: Một danh sách đại diện cho các lựa chọn giới tính.

Dòng 17: Nhằm xác định chuỗi đại diện cho object khi in ra.

2.2.2.1.2. Xây dựng Form xác thực và đăng ký người dùng

```
SearchImage > accounts > forms.py > ...
1 from django import forms
2 from django.contrib.auth.models import User
3 from django.contrib.auth.forms import UserCreationForm
4 from .models import UserProfile
5 from django.core.exceptions import ValidationError
6 import re
7
8 class UserRegistrationForm(UserCreationForm):
9     full_name = forms.CharField(max_length=50, required=True)
10    email = forms.EmailField(required=True)
11    phone = forms.CharField(max_length=15, required=True)
12
13    GENDER_CHOICES=[
14        ('M', 'Male'),
15        ('F', 'Female'),
16        ('N', 'Prefer not to say'),
17    ]
18
19    gender = forms.ChoiceField(choices=GENDER_CHOICES, required=True)
20
21    password1 = forms.CharField(widget=forms.PasswordInput(attrs={'placeholder': 'Enter your password'}), label="Password")
22    password2 = forms.CharField(widget=forms.PasswordInput(attrs={'placeholder': 'Confirm your password'}), label="Confirm Password")
23
```

Click to add a breakpoint

```

25     model = User
26     fields = ['full_name', 'username', 'email', 'phone', 'password1', 'password2', 'gender']
27
28     def clean_username(self):
29         username = self.cleaned_data.get('username')
30         if len(username) < 4:
31             raise ValidationError("Username must be at least 4 characters")
32         return username
33
34     def clean_phone(self):
35         phone = self.cleaned_data.get('phone') or ''
36         phone_numeric = re.sub(r'[-()]\s', '', phone)
37
38         if not re.match(r'^\d{10,15}$', phone_numeric):
39             raise ValidationError("Please enter a valid phone number")
40         return phone
41
42     def clean_password1(self):
43         password = self.cleaned_data.get('password1')
44         if len(password) < 8:
45             raise ValidationError("Password must be at least 8 characters")
46         return password
47
48     def clean_password2(self):
49         password1 = self.cleaned_data.get('password1')
50         password2 = self.cleaned_data.get('password2')
51         if password1 and password2 and password1 != password2:
52             raise ValidationError("Password do not match")
53         return password2

```

```

55     def save(self, commit=True):
56         user = super().save(commit=False)
57         user.email = self.cleaned_data['email']
58
59         full_name = self.cleaned_data['full_name']
60         name_parts = full_name.split(' ', 1)
61         user.first_name = name_parts[0]
62         if len(name_parts) > 1:
63             user.last_name = name_parts[1]
64
65         if commit:
66             user.save()
67
68             UserProfile.objects.create(
69                 user = user,
70                 phone_number = self.cleaned_data['phone'],
71                 gender=self.cleaned_data['gender'],
72             )
73
74         return user
75

```

Đây chính là biểu diễn cho form đăng ký người dùng, được thiết kế với những trường thuộc tính như fullname, email, số điện thoại, danh sách lựa chọn giới tính và nhập password, xác nhận lại mật khẩu.

Dòng 28: Hàm kiểm tra tên người dùng không dưới 4 characters.

Dòng 34: Hàm kiểm tra số điện thoại không chứa ký tự chứa số trong khoảng từ 10 – 15 số.

Dòng 42: Hàm kiểm tra mật khẩu, mật khẩu phải có ít nhất 8 ký tự.

Dòng 48: Hàm xác nhận mật khẩu lần nữa xem có trùng khớp hay không.

Dòng 55: Hàm xác nhận lại trường thuộc tính của người dùng và tạo profile người dùng lưu lại trong database.


```
class LoginForm(forms.Form):
    username = forms.CharField(max_length=150, required=True)
    password = forms.CharField(widget=forms.PasswordInput, required=True)

    def clean_username(self):
        username = self.cleaned_data.get('username')

        if '@' in username:
            try:
                user = User.objects.get(email = username)
                return user.username
            except User.DoesNotExist:
                pass

        return username
```

Form đăng nhập người dùng gồm 2 phần cơ bản chính đó là username và password đã được đăng ký, đồng thời chứa hàm kiểm tra xem người dùng tồn tại hay không

2.2.2.1.3. Triển khai View xử lý logic

```
SearchImage > accounts > views.py > RegisterView
1  from django.shortcuts import render, redirect
2  from django.contrib.auth.models import User
3  from django.contrib.auth import login, authenticate, logout
4  from .models import UserProfile
5  from django.contrib import messages
6  from django.urls import reverse_lazy
7  from django.views import View
8  from django.contrib.auth.views import PasswordResetView, PasswordResetConfirmView, PasswordResetCompleteView, PasswordResetDoneView
9
10 from .forms import UserRegistrationForm, LoginForm
11
12 class RegisterView(View):
13     form_class = UserRegistrationForm
14     template_name = 'accounts/register.html'
15
16     def get(self, request):
17         form = self.form_class()
18         return render(request, self.template_name, {'form':form})
19
20     def post(self, request):
21         form = self.form_class(request.POST)
22
23         if form.is_valid():
24             user = form.save()
25             messages.success(request, 'Registration succesful')
26
27             return redirect('login')
28
29         return render(request, self.template_name, {'form':self.form_class(), 'form_submitted' :True})
30
```

View xử lý đăng ký tài khoản người dùng, khi ấn vào nút ‘Register’ người dùng sẽ được chuyển tiếp tới trang đăng ký.

Dòng 16: Khi người dùng truy cập trang đăng ký (GET request), Django tạo một thể hiện form và render ra template.

Dòng 20: Phương thức POST xử lý người dùng submit form, lấy dữ liệu từ POST và tạo form với dữ liệu đó, song kiểm tra dữ liệu có hợp lệ không.

```

SearchImage > accounts > views.py > LoginView
32 class LoginView(View):
33     form_class = LoginForm
34     template_name = 'accounts/login.html'
35
36     def get(self, request):
37         form = self.form_class()
38         return render(request, self.template_name, {'form':form})
39
40     def post(self, request):
41         form = self.form_class(request.POST)
42
43         if form.is_valid():
44             username = form.cleaned_data['username']
45             password = form.cleaned_data['password']
46             user = authenticate(request, username = username, password = password)
47
48             if user is not None:
49                 login(request, user)
50                 messages.success(request, f'Welcome back, {user.username}')
51                 return redirect('home')
52             else:
53                 messages.error(request, 'Invalid username or password')
54
55         return render(request, self.template_name, {'form':form})

```

View xử lý đăng nhập người dùng, ấn vào nút đăng nhập người dùng sẽ được chuyển tiếp tới trang login.

Dòng 36: Khi người dùng truy cập vào trang đăng nhập, phương thức GET sẽ hiện thị form đăng nhập.

Dòng 40: Hàm POST xử lý đăng nhập, kiểm tra dữ liệu username và password có hợp lệ không.

```
SearchImage > accounts > views.py > LoginView
57 class LogoutView(View):
58     def get(self, request):
60         messages.info(request, 'You have been logged out')
61         return redirect('login')
62
63 class CustomPasswordResetView>PasswordResetView):
64     template_name = 'accounts/password_reset.html'
65     email_template_name = 'accounts/password_reset_email.html'
66     success_url = reverse_lazy('password_reset_done')
67
68 class CustomPasswordResetDoneView>PasswordResetDoneView):
69     template_name = 'accounts/password_reset_done.html'
70
71 class CustomPasswordResetConfirmView>PasswordResetConfirmView):
72     template_name = 'accounts/password_reset_confirm.html'
73     success_url = reverse_lazy('password_reset_complete')
74
75 class CustomPasswordResetCompleteView>PasswordResetCompleteView):
76     template_name = 'accounts/password_reset_complete.html'
```

View xử lý đăng xuất và reset lại mật khẩu, khi /logout hệ thống sẽ có thông báo và chuyển tiếp lại về trang login.

2.2.2.1.4. Xây dựng URL Routing

```
SearchImage > accounts > urls.py > ...
1  from django.urls import path
2  from .views import(
3      RegisterView,
4      LoginView,
5      LogoutView,
6      CustomPasswordResetCompleteView,
7      CustomPasswordResetConfirmView,
8      CustomPasswordResetDoneView,
9      CustomPasswordResetView,
10 )
11
12 urlpatterns = [
13     path('register/', RegisterView.as_view(), name='register'),
14     path('login/', LoginView.as_view(), name='login'),
15     path('logout/', LogoutView.as_view(), name='logout'),
16
17     # Password reset URLs
18     path('password-reset/', CustomPasswordResetView.as_view(), name='password_reset'),
19     path('password-reset/done/', CustomPasswordResetDoneView.as_view(), name='password_reset_done'),
20     path('reset/<uidb64>/<token>', CustomPasswordResetConfirmView.as_view(), name='password_reset_confirm'),
21     path('reset/complete/', CustomPasswordResetCompleteView.as_view(), name='password_reset_complete'),
22 ]
```

Xây dựng url nhằm định nghĩa các đường dẫn URL tương ứng với các View (lớp xử lý logic) để Django biết khi người dùng truy cập vào đường dẫn nào thì phải xử lý như thế nào.

2.2.2.2. Xây dựng Module chính của website (Core)

2.2.2.2.1. Xây dựng Model

```
SearchImage > core > models.py > SearchHistory > Meta
1  from django.db import models
2  from django.contrib.auth.models import User
3
4  class Category(models.Model):
5      name = models.CharField(max_length=100)
6      image_path = models.CharField(max_length=255)
7      alt_text = models.CharField(max_length=200)
8
9      def __str__(self):
10         return self.name
11
12     class Meta:
13         verbose_name_plural = "Categories"
14
```

Thiết kế dùng để phân loại ảnh theo từng danh mục (ví dụ như: Động vật, phong cảnh, hoa,).

```
16 class SearchHistory(models.Model):
17     user = models.ForeignKey(User, on_delete=models.CASCADE, related_name='search_history')
18     query = models.CharField(max_length=255)
19     image_url = models.CharField(max_length=255, blank=True, null=True)
20     image_path = models.CharField(max_length=255, null=True, blank=True)
21     results_count = models.CharField(max_length=255, null=True, blank=True)
22     is_image_search = models.BooleanField(default=False)
23     created_at = models.DateTimeField(auto_now_add=True)
24
25     def __str__(self):
26         return f"{self.user.username} - {self.query}"
27
28     class Meta:
29         verbose_name_plural = "Search History"
30         ordering = ['-created_at']
31
```

Thiết kế nhằm lưu lại thông tin lịch sử người dùng truy xuất ảnh với các trường thông tin như người dùng, thời gian truy xuất, số lượng kết quả nhận được, đường dẫn ảnh truy xuất.

```

33 class SearchResult(models.Model):
34     search_history = models.ForeignKey(SearchHistory, on_delete=models.CASCADE, null=True, blank=True)
35     image_url = models.URLField()
36     image_path = models.CharField(max_length=500, null=True, blank=True)
37     title = models.CharField(max_length=255)
38     source = models.CharField(max_length=100)
39     similarity_score = models.FloatField(null=True, blank=True)
40     metadata = models.JSONField(default=dict)
41     created_at = models.DateTimeField(auto_now_add=True)
42
43     class Meta:
44         ordering = ['-similarity_score']
45
46     def __str__(self):
47         return f"{self.title} - {self.similarity_score}%"
48

```

Lưu lại kết quả của một truy vấn tìm kiếm với những trường thông tin như điểm tương đồng so với ảnh truy vấn, sắp xếp kết quả từ giống nhất đến ít hơn.

```

50 class ImageDataset(models.Model):
51     name = models.CharField(max_length=255)
52     image_path = models.CharField(max_length=500, null=True, blank=True)
53     category = models.ForeignKey(Category, on_delete=models.CASCADE, null=True, blank=True)
54
55     # Metadata từ API
56     source = models.CharField(max_length=50, default='unknown')
57     source_id = models.CharField(max_length=100, null=True, blank=True)
58     source_url = models.URLField(null=True, blank=True)
59
60     # Image properties
61     width = models.IntegerField(null=True, blank=True)
62     height = models.IntegerField(null=True, blank=True)
63     file_size = models.BigIntegerField(null=True, blank=True)
64     format = models.CharField(max_length=10, null=True, blank=True)
65
66     # Content
67     tags = models.TextField(blank=True)
68     description = models.TextField(blank=True)
69
70     # Stats
71     view_count = models.IntegerField(default=0)
72     search_count = models.IntegerField(default=0)
73
74     created_at = models.DateTimeField(auto_now_add=True)
75     updated_at = models.DateTimeField(auto_now=True)
76
77     class Meta:
78         db_table = 'image_dataset'
79         indexes = [
80             models.Index(fields=['category']),
81             models.Index(fields=['source']),
82             models.Index(fields=['created_at']),
83         ]

```

Lớp ImageDataset, tập dữ liệu ảnh được lưu trong dataset như một database để truy xuất ảnh tương tự.

2.2.2.2.2. Xử lý ảnh trước khi truy xuất

```
SearchImage > core > ai_models > image_processor.py > ...
1  import cv2
2  import numpy as np
3  from PIL import Image, ImageEnhance
4  import os
5
6  class ImageProcessor:
7      def __init__(self):
8          pass
9
10     def resize_image(self, image_path, size=(224, 224)):
11         """Resize image to specified size"""
12         try:
13             image = Image.open(image_path)
14             image = image.resize(size, Image.Resampling.LANCZOS)
15             return image
16         except Exception as e:
17             print(f"Error resizing image {image_path}: {e}")
18             return None
19
20     def enhance_image(self, image_path, enhance_factor=1.2):
21         """Enhance image contrast and sharpness"""
22         try:
23             image = Image.open(image_path)
24
25             # Enhance contrast
26             enhancer = ImageEnhance.Contrast(image)
27             image = enhancer.enhance(enhance_factor)
28
29             # Enhance sharpness
30             enhancer = ImageEnhance.Sharpness(image)
31             image = enhancer.enhance(enhance_factor)
32
33             return image
34         except Exception as e:
35             print(f"Error enhancing image {image_path}: {e}")
36             return None
37
```

Dòng 10: Hàm `resize_image` nhằm đưa ảnh về định dạng cố định ở đây là `size = 224 x 224` phù hợp đối với mô hình ResNet – 50. Sử dụng “PIL.Image” và thuật toán “LANCZOS” để lấy chất lượng cao nhất khi resize.

Dòng 20: Làm tăng chất lượng của ảnh trước khi extract đặc trưng, tăng độ tương phản và độ sắc nét của ảnh.


```

38     def extract_color_histogram(self, image_path, bins=32):
39         """Extract color histogram features"""
40         try:
41             image = cv2.imread(image_path)
42             image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
43
44             # Calculate histogram for each channel
45             hist_r = cv2.calcHist([image], [0], None, [bins], [0, 256])
46             hist_g = cv2.calcHist([image], [1], None, [bins], [0, 256])
47             hist_b = cv2.calcHist([image], [2], None, [bins], [0, 256])
48
49             # Concatenate histograms
50             hist = np.concatenate([hist_r, hist_g, hist_b]).flatten()
51             hist = hist / np.sum(hist) # Normalize
52
53             return hist
54         except Exception as e:
55             print(f"Error extracting color histogram from {image_path}: {e}")
56             return None
57
58     def extract_text_features(self, image_path):

```

Dòng 38: Trích xuất đặc trưng của màu ảnh dưới dạng histogram, tính histogram cho 3 màu kênh riêng biệt (R, G, B) xong ghép và chuẩn hóa histogram thành các vecto đặc trưng.

```

58 def extract_texture_features(self, image_path):
59     """Extract texture features using LBP"""
60     try:
61         image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
62
63         # Local Binary Pattern
64         def lbp(image, radius=1, neighbors=8):
65             h, w = image.shape
66             lbp_image = np.zeros((h, w), dtype=np.uint8)
67
68             for i in range(radius, h - radius):
69                 for j in range(radius, w - radius):
70                     center = image[i, j]
71                     binary_string = ""
72
73                     for k in range(neighbors):
74                         angle = 2 * np.pi * k / neighbors
75                         x = int(i + radius * np.cos(angle))
76                         y = int(j + radius * np.sin(angle))
77
78                         if image[x, y] >= center:
79                             binary_string += "1"
80                         else:
81                             binary_string += "0"
82
83                     lbp_image[i, j] = int(binary_string, 2)
84
85             return lbp_image
86
87         lbp_image = lbp(image)
88
89         # Calculate histogram of LBP values
90         hist, _ = np.histogram(lbp_image, bins=256, range=(0, 256))
91         hist = hist / np.sum(hist) # Normalize
92
93         return hist
94     except Exception as e:
95         print(f"Error extracting texture features from {image_path}: {e}")
96         return None

```

Dòng 58: Trích xuất đặc trưng về kết cấu bằng LBP nhằm so sánh pixel trung tâm với các pixel xung quanh dựa trên bán kính.

Dòng 68: Gán giá trị trung tâm tại vị trí [i, j], so sánh các giá trị của pixel lân cận neighbours. Mỗi pixel sinh ra một chuỗi nhị phân, mã hóa thành giá trị số nguyên.

Dòng 90: Histogram biểu diễn tần suất xuất hiện của các giá trị LBP, chuẩn hóa thành các vectơ đặc trưng với length = 256.

```
98  def get_image_metadata(self, image_path):
99      """Get image metadata"""
100     try:
101         image = Image.open(image_path)
102
103         metadata = {
104             'width': image.width,
105             'height': image.height,
106             'format': image.format,
107             'mode': image.mode,
108             'size_bytes': os.path.getsize(image_path)
109         }
110
111         return metadata
112     except Exception as e:
113         print(f"Error getting metadata from {image_path}: {e}")
114         return None
```

Dòng 98: Lấy thông tin cơ bản của ảnh bao gồm chiều cao, chiều rộng, định dạng file, chất lượng màu, dung lượng file, nhằm lưu metadata của ảnh vào database.

2.2.2.2.3. Xây dựng công cụ dòng lệnh trích xuất đặc trưng và chỉ mục

```
SearchImage > core > management > commands > build_index.py > Command > handle
1  from django.core.management.base import BaseCommand
2  from django.conf import settings
3  import os
4  import numpy as np
5  import pickle
6  import faiss
7  from PIL import Image
8  import torch
9  import torch.nn as nn
10 import torchvision.models as models
11 import torchvision.transforms as transforms
12 os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
13
14
15
16 class ImageFeatureExtractor:
17     def __init__(self, device=None):
18         self.device = device or ('cuda' if torch.cuda.is_available() else 'cpu')
19         self.model = self._load_resnet50()
20         self.transform = self._get_transform()
21
22     def _load_resnet50(self):
23         """Load ResNet50 model without final classification layer"""
24         model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)
25         # Remove the final classification layer (fc)
26         model = nn.Sequential(*list(model.children())[:-1])
27         model.eval()
28         model.to(self.device)
29         return model
30
31     def _get_transform(self):
32         """Standard image preprocessing for ResNet50"""
33         return transforms.Compose([
34             transforms.Resize((224, 224)),
35             transforms.ToTensor(),
36             transforms.Normalize(
37                 mean=[0.485, 0.456, 0.406],
38                 std=[0.229, 0.224, 0.225]
39             )
40         ])
```

Dòng 17: Khởi tạo đối tượng, lựa chọn thiết bị tính toán CPU hoặc GPU, tải model cùng transform.

Dòng 22: Tải mô hình ResNet-50 đã huấn luyện trước, loại bỏ lớp phân loại cuối để chỉ giữ lại phần trích xuất đặc trưng.

Dòng 31: Xây dựng pipeline tiền xử lý ảnh cho ResNet-50 (đi từ resize ảnh -> chuyển ảnh sang tensor -> chuẩn hóa ảnh theo mean/ std của ImageNet).

```
SearchImage > core > management > commands > build_index.py > Command > handle
16 class ImageFeatureExtractor:
17     """
18     """
19     def extract_features(self, image_path):
20         """Extract features from a single image"""
21         try:
22             # Load and preprocess image
23             image = Image.open(image_path).convert('RGB')
24             image_tensor = self.transform(image).unsqueeze(0).to(self.device)
25
26             # Extract features
27             with torch.no_grad():
28                 features = self.model(image_tensor)
29                 # Flatten features (ResNet50 outputs [batch_size, 2048, 1, 1])
30                 features = features.flatten().cpu().numpy()
31
32             # L2 normalization
33             features = features / (np.linalg.norm(features) + 1e-8)
34             return features
35
36         except Exception as e:
37             print(f"Error extracting features from {image_path}: {e}")
38             return None
39
40     def extract_features_batch(self, image_paths):
41         """Extract features from a batch of images"""
42         features = []
43         for image_path in image_paths:
44             features.append(self.extract_features(image_path))
45         return features
```

Dòng 42: Trích xuất vecto đặc trưng từ một ảnh duy nhất, với quy trình:

1. Đọc ảnh và chuyển sang màu ảnh RGB.
2. Tiền xử lý ảnh bằng transform.
3. Đưa ảnh qua mô hình nhằm lấy đặc trưng.
4. Biến đổi đầu ra nhiều chiều [1,2048,1,1] thành vecto 1 chiều [2048].
5. Chuẩn hóa L2 (giá trị vecto có độ dài = 1).

```

63 def extract_batch_features(self, image_paths, batch_size=16):
64     """Extract features from multiple images efficiently"""
65     all_features = []
66
67     for i in range(0, len(image_paths), batch_size):
68         batch_paths = image_paths[i:i + batch_size]
69         batch_images = []
70         valid_paths = []
71
72         # Load batch of images
73         for path in batch_paths:
74             try:
75                 image = Image.open(path).convert('RGB')
76                 image_tensor = self.transform(image)
77                 batch_images.append(image_tensor)
78                 valid_paths.append(path)
79             except Exception as e:
80                 print(f"Error loading {path}: {e}")
81                 continue
82
83         if batch_images:
84             # Stack images into batch tensor
85             batch_tensor = torch.stack(batch_images).to(self.device)
86
87             # Extract features for the batch
88             with torch.no_grad():
89                 features = self.model(batch_tensor)
90                 features = features.flatten(1).cpu().numpy() # [batch_size, 2048]
91
92             # L2 normalization for each feature vector
93             for j in range(features.shape[0]):
94                 norm = np.linalg.norm(features[j])
95                 if norm > 1e-8:
96                     features[j] = features[j] / norm
97
98             all_features.extend(features)
99
100     return np.array(all_features) if all_features else np.array([])
101
102

```

Dòng 63: Hàm trích xuất đặc trưng cho nhiều ảnh cùng lúc để tối ưu hóa hiệu năng, với quy trình:

1. Chia ảnh thành những batch nhỏ.
2. Với mỗi batch, đọc và xử lý tensor, stack thành tensor batch.
3. Đưa vào mô hình trích xuất đặc trưng, flatten và chuẩn hóa từng vectơ đầu ra.
4. Ghép tất cả đặc trưng thành mảng lớn.

```

103 class Command(BaseCommand):
104     help = 'Build image search index using ResNet50 features'
105
106     def add_arguments(self, parser):
107         parser.add_argument(
108             '--dataset-path',
109             type=str,
110             default=None,
111             help='Path to image dataset (default: MEDIA_ROOT/dataset)'
112         )
113         parser.add_argument(
114             '--batch-size',
115             type=int,
116             default=16,
117             help='Batch size for processing images'
118         )
119         parser.add_argument(
120             '--rebuild',
121             action='store_true',
122             help='Rebuild entire index from scratch'
123         )
124         parser.add_argument(
125             '--output-dir',
126             type=str,
127             default=None,
128             help='Output directory for index files (default: BASE_DIR/ai_data)'
129         )
130
131     def handle(self, *args, **options):
132         batch_size = options['batch_size']
133         rebuild = options['rebuild']
134
135         # Determine dataset path
136         if options['dataset_path']:
137             dataset_path = options['dataset_path']
138         else:
139             dataset_path = os.path.join(settings.MEDIA_ROOT, 'dataset')
140
141         # Determine output directory
142         if options['output_dir']:
143             ai_data_path = options['output_dir']
144         else:
145             ai_data_path = os.path.join(settings.BASE_DIR, 'ai_data')
146
147         self.stdout.write(f"Dataset path: {dataset_path}")
148         self.stdout.write(f"Output directory: {ai_data_path}")

```

Xây dựng lớp command – câu lệnh để xử lý việc trích xuất đặc trưng ở trên và lưu lại vào thư mục ai_data của bài tập.

2.2.2.2.4. Xây dựng công cụ tìm kiếm ảnh tương tự

```
SearchImage > core > ai_models > similar_seachimage.py > SimilarityEngine > _load_index
56
57 class SimilarityEngine:
58     def __init__(self, ai_data_path=None):
59         self.index = None
60         self.mappings = None
61         self.extractor = None
62         self.ai_data_path = ai_data_path or os.path.join(settings.BASE_DIR, 'ai_data')
63         self._load_index()
64
65     def _load_index(self):
66         """Load the pre-built search index"""
67         try:
68             # Load FAISS index
69             index_path = os.path.join(self.ai_data_path, 'image_index_resnet50.faiss')
70             if os.path.exists(index_path):
71                 self.index = faiss.read_index(index_path)
72                 print(f" Loaded FAISS index from {index_path}")
73             else:
74                 print(f" Index file not found: {index_path}")
75                 return False
76
77             # Load mappings
78             mappings_path = os.path.join(self.ai_data_path, 'mappings_resnet50.pkl')
79             if os.path.exists(mappings_path):
80                 with open(mappings_path, 'rb') as f:
81                     self.mappings = pickle.load(f)
82                 print(f" Loaded mappings from {mappings_path}")
83             else:
84                 print(f" Mappings file not found: {mappings_path}")
85                 return False
86
87             # Initialize feature extractor only when index is loaded successfully
88             self.extractor = ImageFeatureExtractor()
89             print(" Feature extractor initialized")
90
91             return True
92
93         except Exception as e:
94             print(f" Error loading index: {e}")
95             return False
96
```

Dòng 58: Hàm khởi tạo, tạo đường dẫn tới thư mục chứa dữ liệu deer tại index của FAISS và mappings.

Dòng 65: Tải chỉ mục tìm kiếm và ánh xạ từ file.

Dòng 70: Đường dẫn chứa FAISS index đã được huấn luyện.

Dòng 78: Đường dẫn mappings chứa danh sách đường dẫn ảnh, tên ảnh, đặc trưng, models.


```

57 class SimilaritySearchEngine:
97     def is_ready(self):
98         """Check if the search engine is ready to use"""
99         return (self.index is not None and
100                 self.mappings is not None and
101                 self.extractor is not None)
102
103     def search(self, query_image_path, top_k=10, score_threshold=0.1):
104         """Search for similar images"""
105         if not self.is_ready():
106             print(" Search engine not ready. Please build index first.")
107             return []
108
109         # Extract features from query image
110         query_features = self.extractor.extract_features(query_image_path)
111
112         if query_features is None:
113             print(f" Could not extract features from {query_image_path}")
114             return []
115
116         # Normalize features for cosine similarity
117         query_features = query_features.reshape(1, -1).astype(np.float32)
118         faiss.normalize_L2(query_features)
119
120         # Search
121         try:
122             scores, indices = self.index.search(query_features, top_k)
123         except Exception as e:
124             print(f" Search error: {e}")
125             return []
126
127         # Filter results by threshold and validate indices
128         results = []
129         image_paths = self.mappings.get('image_paths', [])
130         image_names = self.mappings.get('image_names', [])
131
132         for score, idx in zip(scores[0], indices[0]):
133             # Skip invalid indices
134             if idx < 0 or idx >= len(image_paths):
135                 continue
136
137             # Skip results below threshold
138             if score < score_threshold:
139                 continue

```

Dòng 97: Kiểm tra hệ thống đã sẵn sàng để tìm kiếm chưa.

Dòng 103: Hàm search tìm top ảnh tương tự truy vấn.

Dòng 110: Trích xuất đặc trưng từ ảnh truy vấn.

Dòng 117: Gọi tới index FAISS chuẩn hóa vecto theo thuật toán L2.

Dòng 127: Thực hiện lọc kết quả theo score_threshold và kiểm tra xem ảnh có tồn tại hay không. Trả về danh sách tương tự.

```

57 class SimilaritySearchEngine:
103     def search(self, query_image_path, top_k=10, score_threshold=0.1):
140
141         # Check if image file still exists
142         image_path = image_paths[idx]
143         if not os.path.exists(image_path):
144             print(f" Image not found: {image_path}")
145             continue
146
147         results.append({
148             'image_path': image_path,
149             'image_name': image_names[idx],
150             'similarity_score': float(score),
151             'model': 'ResNet50',
152             'index': int(idx)
153         })
154
155     return results
156
157 def search_by_features(self, query_features, top_k=10, score_threshold=0.1):
158     """Search using pre-extracted features"""
159     if not self.is_ready():
160         print(" Search engine not ready. Please build index first.")
161         return []
162
163     # Normalize features
164     query_features = query_features.reshape(1, -1).astype(np.float32)
165     faiss.normalize_L2(query_features)
166
167     # Search
168     try:
169         scores, indices = self.index.search(query_features, top_k)
170     except Exception as e:
171         print(f" Search error: {e}")
172         return []
173
174     # Process results
175     results = []
176     image_paths = self.mappings.get('image_paths', [])
177     image_names = self.mappings.get('image_names', [])
178
179     for score, idx in zip(scores[0], indices[0]):
180         if idx < 0 or idx >= len(image_paths) or score < score_threshold:
181             continue
182

```

Dòng 157: Hàm search ảnh theo đặc trưng đã có sẵn không cần ảnh, phù hợp cho việc truy vấn ảnh đã tìm kiếm từ trước.

```

183         results.append({
184             'image_path': image_paths[idx],
185             'image_name': image_names[idx],
186             'similarity_score': float(score),
187             'model': 'ResNet50',
188             'index': int(idx)
189         })
190
191     return results
192
193     def get_stats(self):
194         """Get index statistics"""
195         if not self.is_ready():
196             return {
197                 'status': 'not_ready',
198                 'message': 'Index not loaded. Please build index first.'
199             }
200
201         return {
202             'status': 'ready',
203             'total_images': self.index.ntotal,
204             'features_dimension': self.mappings.get('feature_dimension', 2048),
205             'model': self.mappings.get('model_name', 'ResNet50'),
206             'dataset_path': self.mappings.get('dataset_path', 'Unknown'),
207             'index_type': 'IVF' if hasattr(self.index, 'nlist') else 'Flat',
208             'device': self.extractor.device if self.extractor else 'Unknown'
209         }
210

```

Dòng 193: Hàm trả về thông tin index bao gồm tổng số ảnh, chiều của vector mặc định là 2048, model, dataset, device sử dụng CPU hoặc GPU.

2.2.2.2.5. Xây dựng View xử lý logic

```
SearchImage > core > views.py > ...
1  from django.shortcuts import render, redirect
2  from django.contrib.auth.decorators import login_required
3  from django.contrib import messages
4  from django.views import View
5  from django.utils.decorators import method_decorator
6  from django.core.paginator import Paginator
7  from django.http import JsonResponse
8  from django.conf import settings
9  from .models import Category, SearchHistory, SearchResult, ImageDataset
10 from .ai_models.similar_seachimage import SimilaritySearchEngine
11 from .ai_models.image_processor import ImageProcessor
12 import os
13 import uuid
14 import logging
15
16 logger = logging.getLogger(__name__)
17
18 class HomeView(View):
19     def get(self, request):
20         categories = Category.objects.all()
21
22         recent_searches = None
23         if request.user.is_authenticated:
24             recent_searches = SearchHistory.objects.filter(user=request.user).order_by('-created_at')[:5]
25
26         # Initialize search engine and get stats
27         search_engine = SimilaritySearchEngine()
28         index_stats = search_engine.get_stats()
29
30         context = {
31             'categories': categories,
32             'recent_searches': recent_searches,
33             'index_stats': index_stats,
34         }
35
36         return render(request, 'core/home.html', context)
37
```

HomeView xử lý hiển thị trang chủ với đầy đủ thông tin từ việc tìm kiếm ảnh tương tự tới thống kê lịch sử tìm kiếm của người dùng khi đã đăng nhập.

```

39 @method_decorator(login_required, name='dispatch')
40 class SearchByImageView(View):
41     def post(self, request):
42         logger.info("Starting image search...")
43
44         if 'image' not in request.FILES:
45             messages.error(request, "Vui lòng tải lên một hình ảnh.")
46             return redirect('home')
47
48         image = request.FILES['image']
49         logger.info(f"Uploaded image: {image.name}, size: {image.size}")
50
51         # Validate file
52         if not self._validate_image_file(image):
53             return redirect('home')
54
55         try:
56             # Save uploaded image
57             file_path = self._save_uploaded_image(image)
58             if not file_path:
59                 return redirect('home')
60
61             logger.info(f"Image saved to: {file_path}")
62
63             # Initialize search engine
64             search_engine = self._initialize_search_engine()
65             if not search_engine:
66                 return redirect('home')
67
68             # Perform search
69             results = self._perform_search(search_engine, file_path)
70             if not results:
71                 messages.warning(request, "Không tìm thấy ảnh tương tự. Hãy thử với ảnh khác hoặc giảm ngưỡng tương đồng.")
72                 return redirect('home')
73
74             # Get image metadata
75             image_info = self._get_image_metadata(file_path)
76
77             # Save search history
78             search_history = self._save_search_history(request.user, file_path, len(results))
79
80             # Process results for display
81             processed_results = self._process_results(results, search_history)
82

```

```

SearchImage > core > views.py > ...
40 class SearchByImageView(View):
41     def post(self, request):
42
43         # Prepare context and render
44         return self._render_results(request, page_obj, file_path, image_info,
45                                     len(processed_results), search_history, image.name)
46
47     except Exception as e:
48         logger.error(f"Unexpected error in search: {e}")
49         messages.error(request, f"Bà xảy ra lỗi không mong muốn: {str(e)}")
50         return redirect('home')
51
52     def _validate_image_file(self, image):
53         """Validate uploaded image file"""
54         valid_extensions = ['.jpg', '.jpeg', '.png', '.gif', '.webp', '.bmp', '.tiff']
55         ext = os.path.splitext(image.name)[1].lower()
56
57         if ext not in valid_extensions:
58             messages.error(self.request, "Định dạng file không được hỗ trợ. Vui lòng tải lên JPG, PNG, GIF, WEBP, BMP hoặc TIFF.")
59             return False
60
61         if image.size > 15 * 1024 * 1024: # 15MB
62             messages.error(self.request, "File quá lớn. Kích thước tối đa là 15MB.")
63             return False
64
65         return True
66
67     def _save_uploaded_image(self, image):
68         """Save uploaded image to disk"""
69         try:
70             ext = os.path.splitext(image.name)[1].lower()
71             filename = f"{uuid.uuid4().hex}{ext}"
72
73             search_images_dir = os.path.join(settings.MEDIA_ROOT, 'search_images')
74             os.makedirs(search_images_dir, exist_ok=True)
75
76             file_path = os.path.join(search_images_dir, filename)
77
78             with open(file_path, 'wb+') as destination:
79                 for chunk in image.chunks():
80                     destination.write(chunk)
81
82             return file_path
83         except Exception as e:
84             logger.error(f"Error saving image: {e}")
85             messages.error(self.request, "Lỗi khi lưu ảnh.")
86             return None

```

View xử lý việc truy xuất ảnh tương tự, trước tiên yêu cầu user phải đăng nhập mới cho phép truy cập.

Dòng 41: Hàm POST xử lý upload ảnh, kiểm tra ảnh từ định dạng tới kích thước hợp lệ thì lưu lại tại server.

Thực hiện gọi đến từng hàm tiếp theo để phục vụ cho việc tìm kiếm ảnh tương tự.

```
SearchImage > core > views.py > ...
40 class SearchByImageView(View):
131 def _initialize_search_engine(self):
132     try:
133         search_engine = SimilaritySearchEngine()
134
135         # Check if search engine is ready
136         if not search_engine.is_ready():
137             logger.error("Search engine not ready - index not found")
138             messages.error(self.request, "Công cụ tìm kiếm không khả dụng. Vui lòng đảm bảo đã xây dựng index bằng: python manage.py build_image_index")
139             return None
140
141         logger.info("Search engine initialized and ready")
142         return search_engine
143     except Exception as e:
144         logger.error(f"Error initializing search engine: {e}")
145         messages.error(self.request, f"Lỗi khởi tạo công cụ tìm kiếm: {str(e)}")
146         return None
147
148 def _perform_search(self, search_engine, file_path):
149     """Perform similarity search"""
150     try:
151         logger.info("Starting similarity search...")
152
153         # Get similarity threshold from request or use default
154         score_threshold = float(self.request.POST.get('threshold', 0.1))
155         top_k = int(self.request.POST.get('top_k', 20))
156
157         results = search_engine.search(
158             file_path,
159             top_k=top_k,
160             score_threshold=score_threshold
161         )
162
163         logger.info(f"Search completed, found {len(results)} results")
164         return results
165     except Exception as e:
166         logger.error(f"Error during search: {e}")
167         messages.error(self.request, f"Lỗi trong quá trình tìm kiếm: {str(e)}")
168         return None
169
```

Trước tiên gọi tới hàm khởi tạo `search_engine`, tại hàm `search_engine` gọi đến công cụ đã được tạo đồng thời kiểm tra xem công cụ đã sẵn sàng hay chưa, có bị lỗi hay không.

Tiếp theo, thực hiện tìm kiếm tại hàm `perform`, khởi tạo luồng lấy tham số từ HTTP request với ngưỡng điểm tối đa và kết quả trả về tối đa mặc định là 20. Gọi phương thức `search` để thực hiện tìm kiếm với các tham số đã lấy, xử lý kết quả được trả về.

```

SearchImage > core > views.py > ...
40  class SearchByImageView(View):
170
171      def _get_image_metadata(self, file_path):
172          """Get image metadata"""
173          try:
174              image_processor = ImageProcessor()
175              metadata = image_processor.get_image_metadata(file_path)
176
177              # Add file size info
178              if metadata:
179                  file_size = os.path.getsize(file_path)
180                  metadata['file_size_mb'] = round(file_size / (2048 * 2048), 2)
181
182              return metadata
183          except Exception as e:
184              logger.error(f"Error getting image metadata: {e}")
185              return None
186
187      def _save_search_history(self, user, file_path, results_count):
188          """Save search history"""
189          try:
190              # Create relative path for storage
191              relative_path = os.path.relpath(file_path, settings.MEDIA_ROOT)
192
193              search_history = SearchHistory.objects.create(
194                  user=user,
195                  query="Image Search",
196                  is_image_search=True,
197                  image_path=relative_path, # Store relative path
198                  results_count=results_count
199              )
200              logger.info(f"Search history saved with ID: {search_history.id}")
201              return search_history
202          except Exception as e:
203              logger.error(f"Error saving search history: {e}")
204              return None

```

Hàm xử lý lấy meta data (thông tin mô tả) của một file hình ảnh và lưu vào lịch sử tìm kiếm vào cơ sở dữ liệu.

```

SearchImage > core > views.py > ...
40 class SearchByImageView(View):
205
206 def _process_results(self, results, search_history):
207     """Process results for display"""
208     processed_results = []
209
210     for i, result in enumerate(results):
211         try:
212             # Convert absolute path to relative URL
213             relative_path = os.path.relpath(result['image_path'], settings.MEDIA_ROOT)
214             image_url = f"/media/{relative_path.replace(os.sep, '/')}"
215
216             # Extract additional info from image name/path
217             image_name = result['image_name']
218             image_title = os.path.splitext(image_name)[0].replace('_', ' ').title()
219
220             processed_result = {
221                 'image_url': image_url,
222                 'image_path': result['image_path'],
223                 'image_name': image_name,
224                 'title': image_title,
225                 'source': 'Dataset',
226                 'similarity': result['similarity_score'],
227                 'similarity_percent': int(result['similarity_score'] * 100),
228                 'model': result.get('model', 'ResNet50'),
229                 'index': result.get('index', i),
230                 'metadata': {
231                     'model': result.get('model', 'ResNet50'),
232                     'feature_extraction': 'CNN Deep Learning'
233                 }
234             }
235             processed_results.append(processed_result)
236
237             # Save individual search result
238             if search_history:
239                 self._save_search_result(search_history, processed_result)
240
241         except Exception as e:
242             logger.error(f"Error processing result {i}: {e}")
243             continue
244
245     return processed_results
246

```

Phương này dùng để xử lý chuyển đổi kết quả thô từ hệ thống tìm kiếm thành định dạng chuẩn để hiển thị trên giao diện người dùng.


```

SearchImage > core > views.py > ...
40  class SearchByImageView(View):
247  def _save_search_result(self, search_history, result):
248      """Save individual search result"""
249      try:
250          SearchResult.objects.create(
251              search_history=search_history,
252              image_url=result['image_url'],
253              image_path=result['image_path'],
254              title=result['title'],
255              source=result['source'],
256              similarity_score=result['similarity'],
257              metadata=result['metadata']
258          )
259      except Exception as e:
260          logger.error(f"Error saving search result: {e}")
261
262  def _setup_pagination(self, request, results):
263      """Setup pagination for results"""
264      results_per_page = int(request.GET.get('per_page', 50))
265      paginator = Paginator(results, results_per_page)
266      page_number = request.GET.get('page', 1)
267      return paginator.get_page(page_number)
268
269  def _render_results(self, request, page_obj, file_path, image_info,
270                      total_results, search_history, original_name):
271      """Render search results template"""
272      filename = os.path.basename(file_path)
273
274      context = {
275          'search_results': page_obj,
276          'original_image': f"/media/search_images/{filename}",
277          'image_info': image_info,
278          'total_results': total_results,
279          'search_history_id': search_history.id if search_history else None,
280          'query_image_name': original_name,
281          'has_results': total_results > 0,
282      }
283
284      logger.info("Rendering search results template")
285      return render(request, 'core/search_results.html', context)
286

```

Phương thức nhằm lưu lại kết quả tìm kiếm theo format và render kết quả nhận được lên trang kết quả của website.

```

SearchImage > core > views.py > DeleteSearchHistoryView > delete
308 @method_decorator(login_required, name='dispatch')
309 class SearchHistoryView(View):
310     def get(self, request):
311         # Lấy tất cả lịch sử tìm kiếm của user, ưu tiên image search
312         histories = SearchHistory.objects.filter(
313             user=request.user
314         ).order_by('-created_at')
315
316         # Xử lý dữ liệu để hiển thị
317         processed_histories = []
318         for history in histories:
319             processed_history = {
320                 'id': history.id,
321                 'query': history.query,
322                 'is_image_search': history.is_image_search,
323                 'created_at': history.created_at,
324                 'results_count': history.results_count,
325                 'image_url': None,
326                 'image_path': history.image_path
327             }
328
329             # Nếu là image search và có image_path
330             if history.is_image_search and history.image_path:
331                 try:
332                     # Tạo URL cho hình ảnh
333                     if not history.image_path.startswith('/media/'):
334                         # Nếu là relative path
335                         image_url = f"/media/{history.image_path.replace(os.sep, '/')}"
336                     else:
337                         # Nếu đã là full path
338                         relative_path = os.path.relpath(history.image_path, settings.MEDIA_ROOT)
339                         image_url = f"/media/{relative_path.replace(os.sep, '/')}"
340
341                     processed_history['image_url'] = image_url
342
343                     # Kiểm tra file có tồn tại không
344                     if history.image_path.startswith('/'):
345                         full_path = history.image_path
346                     else:
347                         full_path = os.path.join(settings.MEDIA_ROOT, history.image_path)
348
349                     processed_history['image_exists'] = os.path.exists(full_path)
350
351                 except Exception as e:
352                     logger.error(f"Error processing image path for history {history.id}: {e}")
353                     processed_history['image_exists'] = False
354

```

Hàm xử lý hiển thị lịch sử tìm kiếm của người dùng render hiển thị ra trang history của người dùng.

```

371 @method_decorator(login_required, name='dispatch')
372 class DeleteSearchHistoryView(View):
373     """API để xóa lịch sử tìm kiếm"""
374     def delete(self, request, history_id):
375         try:
376             history = SearchHistory.objects.get(
377                 id=history_id,
378                 user=request.user
379             )
380
381             # Xóa file ảnh nếu là image search
382             if history.is_image_search and history.image_path:
383                 try:
384                     if history.image_path.startswith('/'):
385                         file_path = history.image_path
386                     else:
387                         file_path = os.path.join(settings.MEDIA_ROOT, history.image_path)
388
389                     if os.path.exists(file_path):
390                         os.remove(file_path)
391                         logger.info(f"Deleted image file: {file_path}")
392                 except Exception as e:
393                     logger.error(f"Error deleting image file: {e}")
394
395             # Xóa search history
396             history.delete()
397
398             return JsonResponse({
399                 'status': 'success',
400                 'message': 'Đã xóa lịch sử tìm kiếm thành công.'
401             })
402
403         except SearchHistory.DoesNotExist:
404             return JsonResponse({
405                 'status': 'error',
406                 'message': 'Không tìm thấy lịch sử tìm kiếm.'
407             }, status=404)
408         except Exception as e:
409             logger.error(f"Error deleting search history: {e}")
410             return JsonResponse({
411                 'status': 'error',
412                 'message': str(e)
413             }, status=500)
414

```

Hàm xử lý API xóa lịch sử tìm kiếm người dùng.

2.2.2.2.6. Triển khai URL Routing

```
SearchImage > core > urls.py > ...
1  from django.urls import path
2  from . import views
3
4  urlpatterns = [
5      path('', views.HomeView.as_view(), name='home'),
6      path('search/image/', views.SearchByImageView.as_view(), name='search_by_image'),
7      path('search/text/', views.TextSearchView.as_view(), name='text_search'),
8      path('find-similar/<int:image_id>', views.FindSimilarView.as_view(), name='find_similar'),
9      path('search-history/', views.SearchHistoryView.as_view(), name='search_history'),
10
11     # API endpoints
12     path('api/index/stats/', views.IndexStatsView.as_view(), name='index_stats'),
13     path('api/index/rebuild/', views.RebuildIndexView.as_view(), name='rebuild_index'),
14     path('api/results/load-more/', views.LoadMoreResultsView.as_view(), name='load_more_results'),
15     path('api/delete-search-history/<int:history_id>', views.DeleteSearchHistoryView.as_view(), name='delete_search_history'),
16
17 ]
```

Xây dựng url nhằm định nghĩa các đường dẫn URL tương ứng với các View (lớp xử lý logic). Để Server biết khi người dùng truy cập vào đường dẫn thì phải xử lý như thế nào.

2.3. Kết luận

Chương 2 đã trình bày chi tiết quá trình thiết kế và triển khai một hệ thống tìm kiếm ảnh thông minh hoàn chỉnh, tích hợp các công nghệ tiên tiến để tạo ra một giải pháp tìm kiếm ảnh tương tự hiệu quả và thân thiện với người dùng.

Về mặt kiến trúc hệ thống, việc lựa chọn và tối ưu hóa mô hình ResNet-50 đã chứng minh tính hiệu quả trong việc trích xuất đặc trưng ảnh với độ chính xác cao. Mô hình này không chỉ đảm bảo chất lượng trong việc nhận diện và phân loại hình ảnh mà còn duy trì hiệu suất xử lý phù hợp ứng dụng thời gian thực. Sự kết hợp với thư viện FAISS đã giải quyết được thách thức về khả năng mở rộng, cho phép hệ thống xử lý và tìm kiếm trên cơ sở dữ liệu ảnh quy mô lớn với tốc độ truy vấn nhanh chóng và hiệu quả về mặt tài nguyên tính toán.

Trong phần triển khai ứng dụng web, kiến trúc frontend đã được thiết kế với trải nghiệm người dùng tối ưu, cung cấp giao diện trực quan và dễ sử dụng cho việc tải lên, tìm kiếm và hiển thị kết quả ảnh. Backend được xây dựng trên nền tảng Django đã thể hiện tính ổn định, các cơ chế xử lý bất đồng bộ và tối ưu hóa cơ sở dữ liệu để đảm bảo hiệu suất ổn định.

Kết quả đạt được từ chương này là một hệ thống tìm kiếm ảnh thông minh hoàn chỉnh, kết hợp giữa công nghệ học sâu tiên tiến và thiết kế ứng dụng web. Hệ thống không chỉ đáp ứng các yêu cầu kỹ thuật về độ chính xác và hiệu suất mà còn mang lại trải nghiệm tốt cho người dùng, đặt nền tảng vững chắc cho việc đánh giá và cải tiến hệ thống trong tương lai.

CHƯƠNG 3: TRIỂN KHAI THỰC NGHIỆM

3.1. Mô hình hệ thống

SearchImage/

```
|— manage.py          # File quản lý Django
|
|— searchimage/       # Thư mục cấu hình chính
|   |— __init__.py
|   |— settings.py    # Cấu hình dự án
|   |— urls.py        # URL routing chính
|   |— asgi.py
|   |— wsgi.py
|
|— accounts/          # Ứng dụng quản lý tài khoản
|   |— __init__.py
|   |— admin.py       # Cấu hình admin
|   |— apps.py        # Cấu hình ứng dụng
|   |— models.py      # Model tài khoản người dùng
|   |— forms.py       # Form đăng nhập/ đăng ký
|   |— urls.py        # URL routing cho accounts
```

```

|   |—— views.py           # Views xử lý logic
|   |—— static/           # File tĩnh cho accounts
|   |   └── templates/    # Templates HTML
|       └── accounts/
|           ├── login.html # Giao diện đăng nhập
|           └── register.html # Giao diện đăng ký
|
|—— core/                # Ứng dụng chính
|   |—— __init__.py
|   |—— admin.py          # Cấu hình admin
|   |—— apps.py           # Cấu hình ứng dụng
|   |—— models.py         # Model dữ liệu chính
|   |—— urls.py           # URL routing cho core
|   |—— views.py          # Views xử lý tìm kiếm
|   |
|   |—— ai_models/        # Module AI/ML
|       |—— __init__.py
|       |—— image_processor.py # Xử lý hình ảnh
|       |   └── similarity_search.py # Tìm kiếm tương tự
|       |

```

```

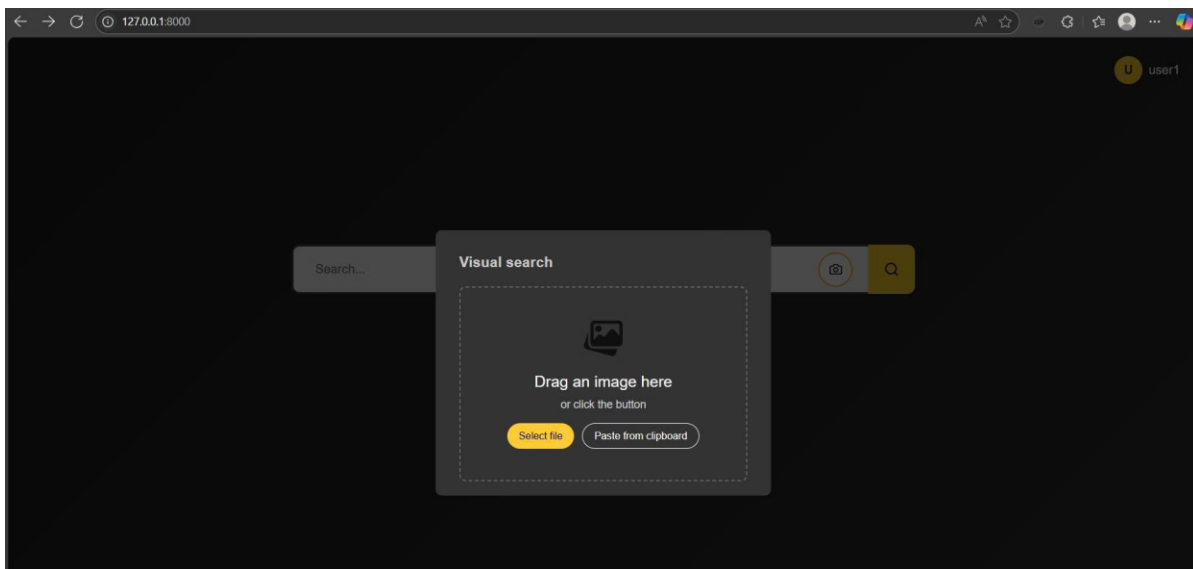
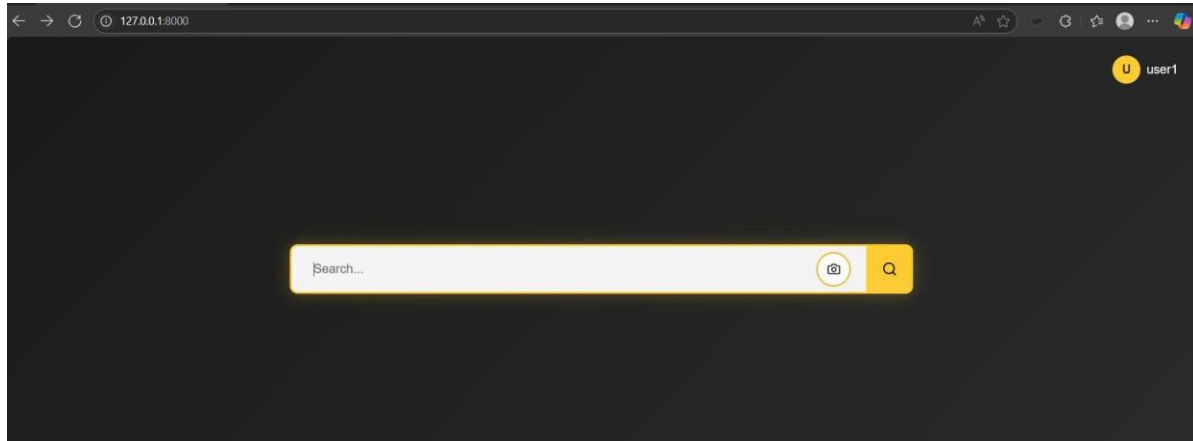
|   └── management/                # Django management commands
|   └── └── commands/
|   └── └── └── __init__.py
|   └── └── └── build_index.py      # Lệnh xây dựng chỉ mục
|   └──
|   └── static/                    # File tĩnh cho core
|   └── └── templates/              # Templates HTML
|   └── └── └── home.html            # Giao diện trang chủ
|   └── └── └── result.html          # Giao diện kết quả tìm kiếm
|   └──
|   └── ai_data/                   # Dữ liệu AI và ML
|   └── └── features/               # Đặc trưng trích xuất
|   └── └── indices/               # Chỉ mục tìm kiếm
|   └── └── models/                # Mô hình AI đã huấn luyện
|   └──
|   └── media/                     # File media upload
|   └── └── dataset/               # Bộ dữ liệu hình ảnh

```


3.2. Triển khai thực nghiệm và đánh giá

3.2.1. Kết quả thực nghiệm

Giao diện trang chủ, bao gồm thanh search và công cụ upload ảnh để tìm kiếm ảnh tương tự.



Kết quả sau khi tìm kiếm cho ra số lượng ảnh tương tự tìm được.

127.0.0.1:8000/search/image/ 20 kết quả

Ảnh tìm kiếm



Tên file: AnhHoa.jpg
Kích thước: 736 x 1177px
Dung lượng: 0.03 MB
Định dạng: JPEG

Độ tương đồng: Tất cả Sắp xếp: Độ tương đồng Mật độ hiển thị: Bình thường

 Abstract 3790834 Dataset 34% tương đồng <a>Xem	 Backgrounds 3790834 Dataset 34% tương đồng <a>Xem	 Backgrounds 8175248 Dataset 40% tương đồng <a>Xem	 Abstract 7228828 Dataset 43% tương đồng <a>Xem	 Backgrounds 7228828 Dataset 43% tương đồng <a>Xem
 Feelings 3825388 Dataset 44% tương đồng <a>Xem	 People 3825388 Dataset 44% tương đồng <a>Xem	 Abstract 8224979 Dataset 52% tương đồng <a>Xem	 Backgrounds 8224979 Dataset 52% tương đồng <a>Xem	 Abstract 7861942 Dataset 55% tương đồng <a>Xem

3.2.2. Kết luận và đánh giá

Qua quá trình thực nghiệm và đánh giá, hệ thống đã được phát triển đã cho thấy những kết quả khả quan trong việc tìm kiếm và truy vấn ảnh dựa trên nội dung thị giác. Hệ thống đã thể hiện khả năng trích xuất và so sánh các đặc trưng ảnh cơ bản, cho phép người dùng thực hiện truy vấn bằng ảnh mẫu và nhận được kết quả tương đối phù hợp. Các thành phần chính của hệ thống bao gồm module tiền xử lý ảnh, trích xuất đặc trưng và tính toán độ tương tự đã hoạt động ổn định. Giao diện người dùng trực quan và dễ sử dụng, cho phép upload ảnh truy vấn và hiển thị kết quả một cách rõ ràng. Thời gian phản hồi của hệ thống nằm trong mức chấp nhận được đối với cơ sở dữ liệu có kích thước vừa phải.

Mặc dù hệ thống có thể trả về kết quả tìm kiếm, độ chính xác vẫn còn những hạn chế nhất định. Trong một số trường hợp, các ảnh được trả về có thể có đặc trưng thị giác tương tự nhưng nội dung ngữ nghĩa hoàn toàn khác biệt. Điều này phản ánh vấn đề semantic gap - khoảng cách giữa đặc trưng thị giác cấp thấp và ngữ nghĩa cấp cao mà con người hiểu được. Hệ thống hiện tại chủ yếu dựa vào các đặc trưng handcrafted truyền thống, có thể không đủ mạnh để nắm bắt được những đặc điểm phức tạp và trừu tượng của ảnh.

Để nâng cao trải nghiệm người dùng và tính hiệu quả của hệ thống, cần phát triển thêm chức năng "tìm kiếm ảnh tương tự mở rộng" (expanded similar image search). Tính năng này sẽ cho phép người dùng không chỉ xem các kết quả top-k mà còn có thể khám phá thêm nhiều ảnh liên quan khác.

Cụ thể, sau khi hiển thị kết quả ban đầu, hệ thống sẽ cung cấp tùy chọn "Xem thêm ảnh tương tự" hoặc "Khám phá thêm" để mở rộng phạm vi tìm kiếm. Điều này có thể được thực hiện bằng cách:

- Tăng số lượng kết quả trả về từ top-5 lên top-20 hoặc top-50
- Cung cấp tùy chọn phân trang để người dùng duyệt qua nhiều trang kết quả
- Thêm tính năng lọc kết quả theo các tiêu chí như màu sắc, kích thước, hoặc loại đối tượng
- Hiển thị các nhóm ảnh tương tự theo cụm (clustering) để người dùng có thể khám phá các hướng tìm kiếm khác nhau

KẾT LUẬN

Hệ thống Content-Based Image Retrieval đã được phát triển thành công đã chứng minh được tính khả thi và tiềm năng ứng dụng trong thực tế. Mặc dù còn những hạn chế nhất định về độ chính xác và phạm vi tìm kiếm, những kết quả đạt được đã đặt nền móng vững chắc cho các bước phát triển tiếp theo.

Việc mở rộng khả năng tìm kiếm và tích hợp đa mô hình không chỉ là những cải tiến kỹ thuật mà còn là những bước tiến quan trọng hướng tới một hệ thống CBIR hoàn thiện và thực dụng. Những đề xuất này phản ánh xu hướng phát triển hiện đại của lĩnh vực computer vision và information retrieval.

Hướng phát triển trong tương lai sẽ tập trung vào việc áp dụng các công nghệ AI tiên tiến nhất, đồng thời duy trì tính thực dụng và hiệu quả của hệ thống. Với sự phát triển không ngừng của deep learning và computer vision, hệ thống CBIR có tiềm năng trở thành một công cụ mạnh mẽ và không thể thiếu trong nhiều lĩnh vực ứng dụng khác nhau.

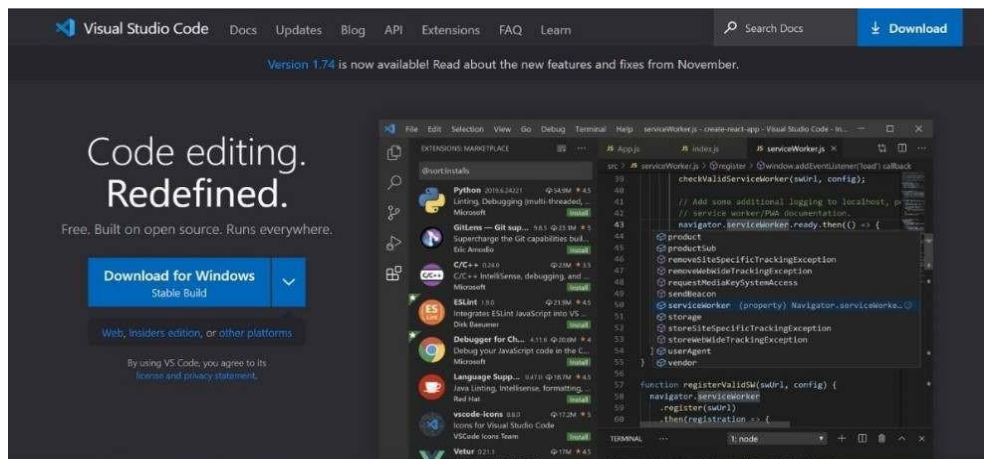
Thành công của dự án này không chỉ nằm ở việc xây dựng được một hệ thống hoạt động, mà còn ở việc tạo ra một framework có thể mở rộng và cải tiến liên tục, sẵn sàng đón nhận những công nghệ mới và đáp ứng những yêu cầu ngày càng cao của người dùng.

PHỤ LỤC

1. Hướng dẫn cài đặt và xây dựng môi trường

1.1. Cài đặt Python

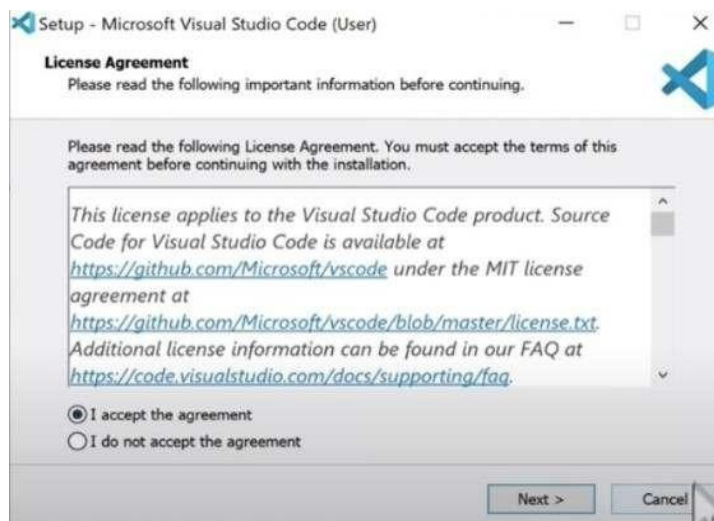
Visual Studio Code có thể download tại website chính thức của Visual Studio:



Truy cập vào trang chủ và chọn hệ điều hành Window

Bước 1: Sau khi tải về thực hiện chọn file để bắt đầu cài đặt.

Bước 2: Chọn I Agree để đồng ý các điều khoản của Visual Studio Code



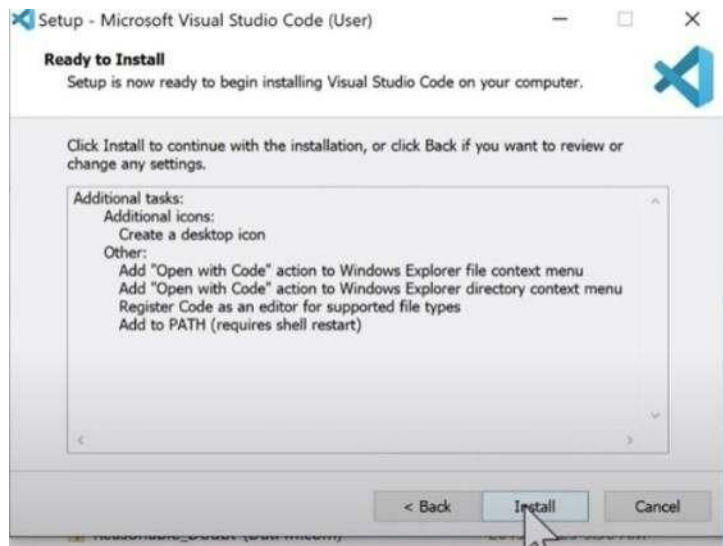
Bước 3: Tích chọn những ô trong Other.



Bước 4: Chọn Next ở lần kế tiếp



Bước 5: Kiểm tra PATH và chọn Install



Bước 6: Chọn Finish để kết thúc việc cài đặt.

Bước 7: Thực hiện cài đặt extension Python



1.2. Cài đặt Django Framework

Django là framework web phát triển bằng Python, được sử dụng để xây dựng ứng dụng web một cách nhanh chóng và hiệu quả.

Thực hiện cài đặt với lệnh:

- `pip install Django`
- `Django-admin --version` (kiểm tra phiên bản của Django).

Thực hiện tạo project mới và môi trường ảo trong Django:

- `python -m venv djangoenv`
- `djangoenv\Scripts\activate` (Kích hoạt môi trường ảo).

Sau khi kích hoạt môi trường ảo, thực hiện tải xuống các package cần thiết:

- `pip install Pillow`
- `pip install opencv-python`
- `pip install numpy`
- `pip install dotenv-python`
- `pip install scikit-learn`
- `pip install tensorflow`
- `pip install requests`
- `pip install beautifulsoup4`

1.3. Build Index để extract đặc trưng vào tạo chỉ mục

Trước tiên, tải xuống các package cần thiết:

- pip `install` torchvision
- pip `install` faiss-cpu

Thực hiện lệnh “`python manage.py build_index`” nhằm xây dựng chỉ mục cho các đặc trưng của ảnh và lưu lại vào thư mục `Project\SearchImage\ai_data`.

1.4. Xây dựng database để lưu ảnh tương tự

Xây dựng thư mục media bao gồm dataset và searchHistory nhằm lưu data ảnh và những hình ảnh đã upload lên.

Khởi chạy server Django:

- Khởi chạy môi trường ảo `djangoenv`
- `python manage.py makemigrations`
- `python manage.py migrate` (khởi tạo bảng và cột để lưu lại vào database).
- `python manage.py createsuperuser` (tạo người dùng admin).
- `python manage.py runserver`

BẢNG PHÂN CÔNG CÔNG VIỆC

Họ và tên - MSV	Công việc
Tạ Tùng Sơn – AT190144	1. Tìm hiểu về cách thức tiền xử lý ảnh. 2. Tìm hiểu về mô hình học sâu và ứng dụng. 3. Triển khai xây dựng ứng dụng thực nghiệm. 4. Xây dựng bản words.
Nguyễn Văn Sơn – AT191043	1. Tìm hiểu về kỹ thuật CBIR xử lý ảnh. 2. Tìm hiểu về thuật toán so sánh đặc trưng của ảnh. 3. Triển khai xây dựng ứng dụng thực nghiệm 4. Xây dựng bản words
Nguyễn Đức Chiến – AT190107	1. Tìm hiểu về các thành phần cơ bản để xây dựng nên thuật toán so sánh. 2. Tích hợp thuật toán so sánh với các mô hình học sâu. 3. Triển khai xây dựng ứng dụng thực nghiệm 4. Xây dựng bản powerpoint

TÀI LIỆU THAM KHẢO

- [1]. Content-Based image retrieval: A review of recent trends.
Online: <https://www.tandfonline.com/doi/full/10.1080/23311916.2021.1927469>
- [2]. Content Based Image Retrieval System Using CNN based Deep Learning Models.
Online: <https://www.sciencedirect.com/science/article/pii/S1877050924009761>
- [3]. Content-Based Image Retrieval and Feature Extraction.
Online: <https://www.hindawi.com/journals/mpe/2019/9658350/>
- [4]. Building an image search engine with python.
Online: <https://thetisdev.hashnode.dev/building-an-image-search-engine-with-python-and-faiss>